

算法设计与分析

授课教师

戴智明

daizhim@mail.sysu.edu.cn

超算中心502B

助教

朱斌

zhub35@mail2.sysu.edu.cn

叶晨辉

yechh7@mail2.sysu.edu.cn

王超

wangch556@mail2.sysu.edu.cn

考核方式

考勤、平时作业
期末考试

40%
60%

课件下载:

课程QQ群

教材：

- 屈婉玲、刘田、张立昂、王捍贫. 算法设计与分析（第3版）. 清华大学出版社

主要参考书：

- T. H. Cormen, C.E. Leiserson, R. L. Rivest, C. Stein. Introduction to Algorithms (4th Edition). MIT Press
- J. Kleinberg, E. Tardos. Algorithm Design. Addison-Wesley
- Robert Sedgewick, Kevin Wayne. Algorithms (4th edition). Addison-Wesley

教材主要内容

近似 算法	随机 算法	处理难解问 题的策略	
NP 完全理论			
算法分析与问题的计算复杂性			
线性规划		网络流算法	
分治 策略	动态 规划	贪心 算法	回溯与 分支限界
数学基础、数据结构			

问题处理策略

计算复杂性理论

算法分析方法

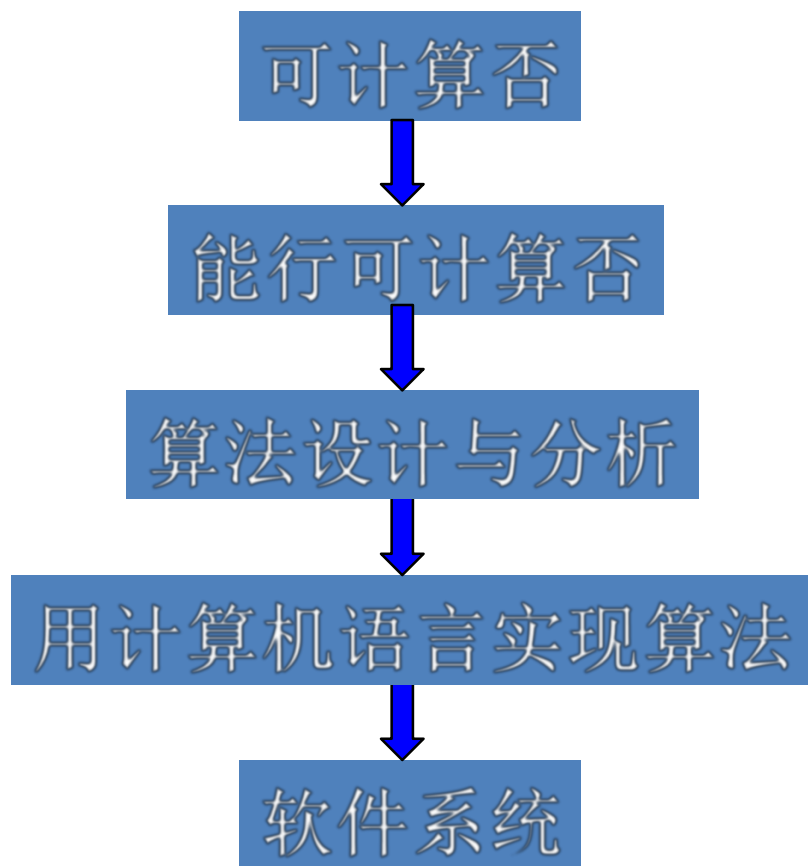
算法设计技术

基础知识

引 论

计算机科学的体系

- 解决一个计算问题的过程



- 可计算理论
 - 计算模型
 - 可计算问题/不可计算问题
 - 计算模型的等价性—图灵/Church命题
- 计算复杂性理论
 - 在给定的计算模型下研究问题的复杂性
 - 固有复杂性
 - 上界
 - 下界
 - 平均
 - 复杂性问题的分类: $P=NP?$
 - 抽象复杂性研究

- 算法设计和分析

- 可计算问题的算法的设计与分析

- 设计算法的理论、方法和技术

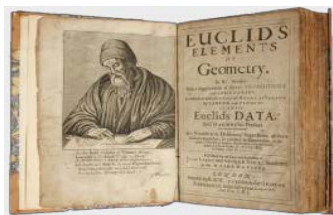
- 如何设计正确高效的算法？

- 分析算法的理论、方法和技术

- 如何判断哪个算法更高效？

设计高效算法，分析算法性能，两者交互验证

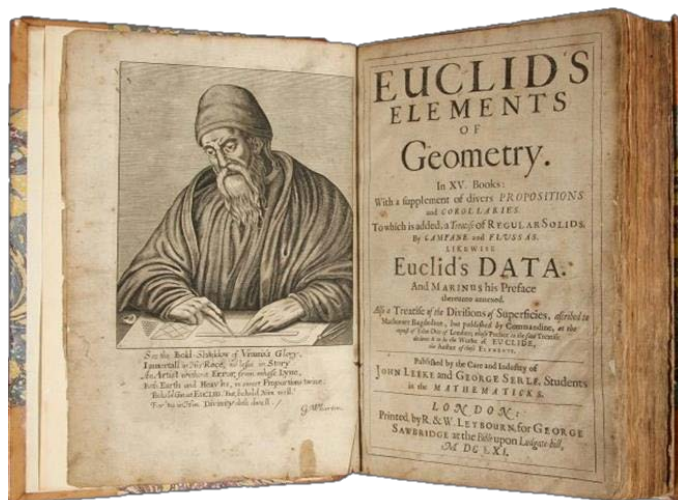
算法的由来



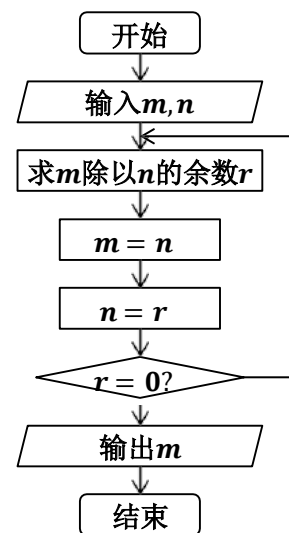
公元前300年
辗转相除法
《几何原本》

算法的由来

- 辗转相除法
 - 用于计算两个整数的最大公约数
 - 约公元前**300**年由欧几里得提出

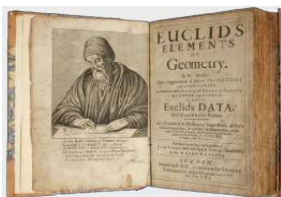


欧几里得《几何原本》



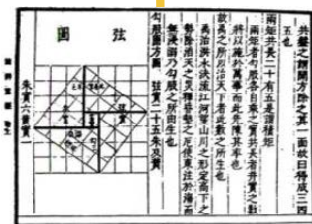
辗转相除法流程图

算法的由来



公元前300年
辗转相除法
《几何原本》

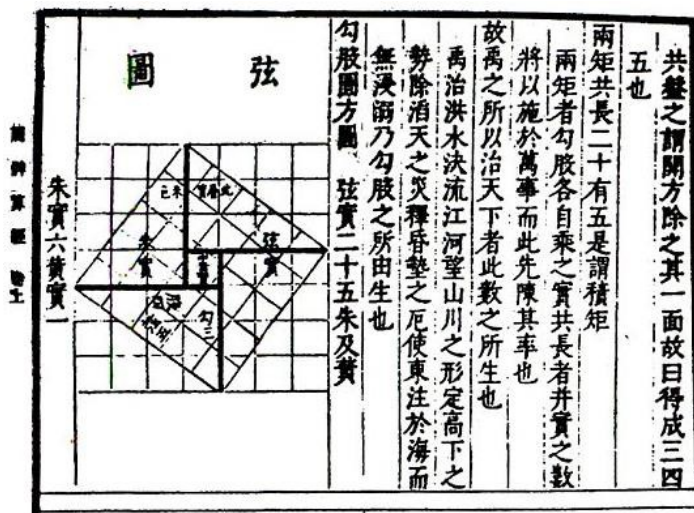
算法
《周髀算经》
公元前1世纪



算法的由来

名称由来

- 中文名称“算法”出自约成书于公元前1世纪的《周髀算经》



介绍了勾股定理：“以日下为勾，日高为股，勾股各自乘，并而开方除之”

欽定四庫全書	子部六
周髀算經	天文算法類一 屬 推步之
提要	
臣	
等謹案周髀算經二卷音義一卷案隋書	
經籍志天文類首列周髀一卷趙嬰注又一	
卷甄鸞重述唐書藝文志李淳風釋周髀二	
卷與趙嬰甄鸞之注列之天文類而復列李	
淳風注周髀算經二卷於歷算類蓋一書重	
出也是書內稱周髀長八尺夏至之日晷一	
尺六寸蓋髀者股也於周地立八尺之表以	
為股其影為勾故曰周髀其首章周公與商	
高相問答實勾股之鼻祖故	
御定數理精蘊載在卷首而詳釋之稱為成周六藝	
之遺文榮方問于陳子以下徐光啟謂為千	
古大愚今詳考其文惟論南北影差以地為	
平遠復以平遠測天誠為臆說然與本文已	

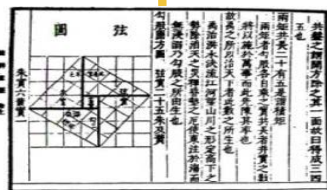
采用简便可行方法确定天文历法
揭示日月星辰运行规律

算法的由来



公元前300年
辗转相除法
《几何原本》

算法
《周髀算经》
公元前1世纪



魏晋时期
割圆术
《九章算术注》

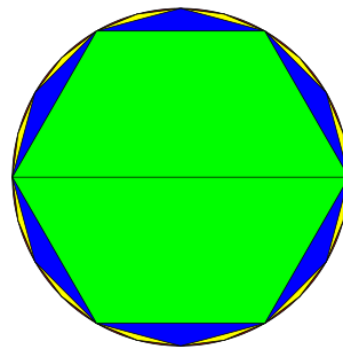
算法的由来

- 割圆术

- 内接正多边形去无限逼近圆，并以此求取圆周率的方法
- 魏晋时期的数学家刘徽在《九章算术注》中首创



刘徽与《九章算术注》



割圆术示意图

算法的由来

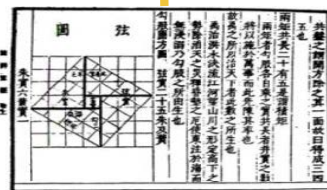


公元前300年
辗转相除法
《几何原本》



7世纪
《算经十书》

算法
《周髀算经》
公元前1世纪



魏晋时期
割圆术
《九章算术注》

算法的由来

算经十书

- 唐高宗显庆元年（公元**656**年），规定将十部汉、唐一千多年间的十部著名数学著作作为国家最高学府的算学教科书，用以进行数学教育和考试，后世通称为《算经十书》



唐高宗李治



周髀算经



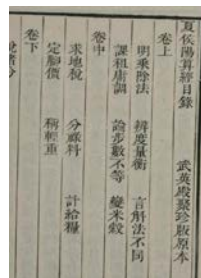
九章算术



海岛算经



张丘建算经



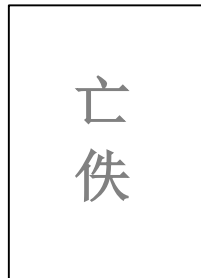
夏侯阳算经



五经算术



缉古算经



缀术



五曹算经



孙子算经

算法的由来

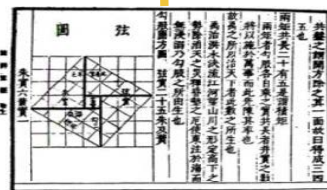


公元前300年
辗转相除法
《几何原本》



7世纪
《算经十书》

算法
《周髀算经》
公元前1世纪



Algorithm
阿尔·花拉子密
9世纪



魏晋时期
割圆术
《九章算术注》

算法的由来

- 名称由来

- 波斯著名的数学家、天文学家、地理学家阿尔·花拉子密在公元**825**年写成《印度数字算术》一书，对于印度-阿拉伯数字系统在中东及欧洲的传播起到了重要作用
- 该书被翻译成拉丁语“**Algoritmi de numero Indorum**”，花拉子密的拉丁文音译即为“算法”（**Algorithm**）一词的由来



苏联在**1983**年发行邮票纪念花拉子密**1200**岁生辰

算法的由来

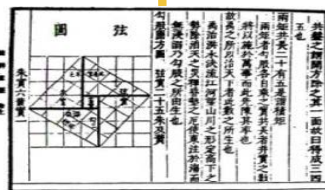


公元前300年
辗转相除法
《几何原本》



7世纪
《算经十书》

算法
《周髀算经》
公元前1世纪



Algorithm
阿尔·花拉子密
9世纪



魏晋时期
割圆术
《九章算术注》

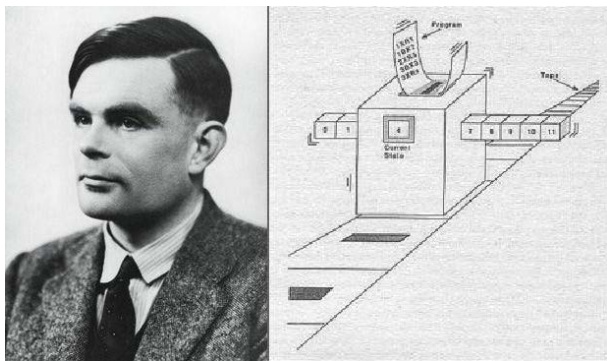


20世纪30年代~40年代
艾伦·图灵与冯·诺依曼

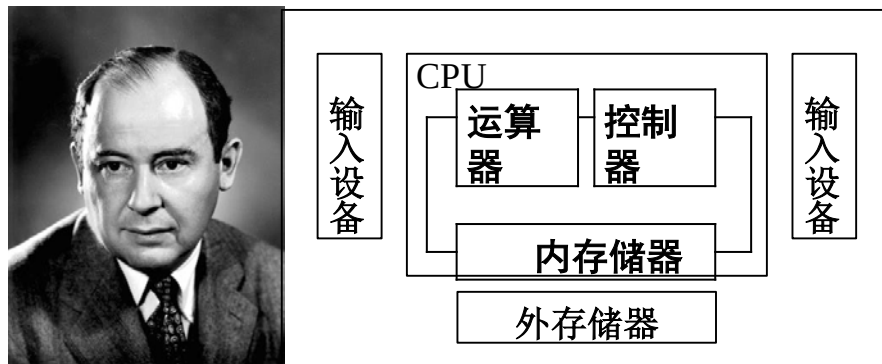
算法的由来

• 算法与计算机的结合

- **1936年**，艾伦·图灵提出**图灵机**，通过建立通用计算机模型，刻画计算机的计算行为
- **1946年**，冯·诺依曼提出**存储程序原理**



理论计算机科学与人工智能之父
艾伦·图灵 **Alan Turing**



现代计算机之父
约翰·冯·诺伊曼 **John von Neumann**

图灵奖

- **1966**年由计算机协会(**ACM**)设立，奖励对计算机事业做出突出贡献的个人



艾伦·图灵
Alan Turing



计算机界的“诺贝尔奖”

在**1966~2023**年的**58**年中，共计**77**位图灵奖获得者

算法与计算复杂性领域图灵奖得主



Donald E. Knuth
1974, USA
算法分析之父



Michael O. Rabin
1976, Israeli
非确定自动机
素数判定随机算法



Dana S. Scott
1976, USA
非确定自动机



Robert W. Floyd
1978, USA
最短路径**Floyd**算法



Stephen A. Cook
1982, USA
NP完全性



Richard M. Karp
1985, USA
NP完全性与
网络流算法



John Hopcroft
1986, USA
最差情况分析
数据结构与算法



Robert Tarjan
1986, USA
数据结构与图算法



Juris Hartmanis
1993, Latvia
计算复杂性理论



Richard E. Stearns
1993, USA
计算复杂性理论



Manuel Blum
1995, Venezuela
计算复杂性理论



Andrew Yao
2000, China
伪随机数生成
与通信复杂性



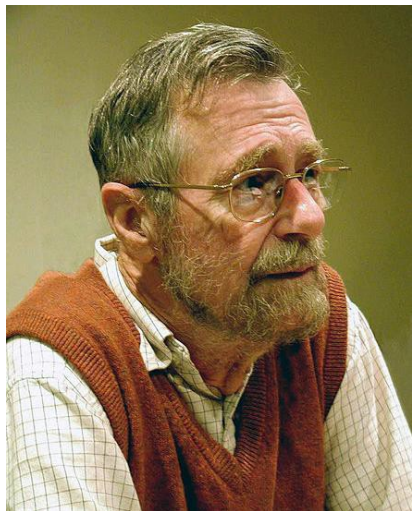
Leslie G. Valiant
2010, Hungarian
#P完全性与
计算学习理论



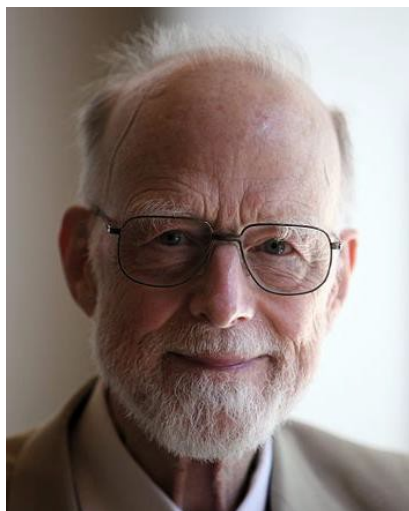
Avi Wigderson
2023, Israeli
随机性与计算复
杂性理论

算法与计算复杂性领域图灵奖得主共计**14**人，占比约**18%**

其他相关图灵奖得主



Edsger W. Dijkstra
1972, Netherlands
ALGOL之父
提出单源最短路径**Dijkstra**算法



Tony Hoare
1980, UK
霍尔逻辑
提出快速排序算法



Alfred Vaino Aho
2020, USA
编程语言实现领域
对基础算法和理论
的贡献



Jeffrey David Ullman
2020, USA
编程语言实现领域对
基础算法和理论的贡
献

算法的定义

算法是一个满足下列条件的计算：

- 输入：有一个满足给定约束条件的输入
- 输出：满足给定约束条件的结果
- 有穷性/终止性：有限步内必须停止
- 确定性：每一步都是严格定义和确定的动作（反例：对于给定输入数组，交换两个数的位置）
- 可行性：每一个动作都能够被精确地机械执行（反例：将大元素放数组后部，小元素放数组前部）

计算机问题求解实例

补丁与Bug问题： 错误就是人们所说的Bug。用户在试用软件时总是希望其错误越少越好，最好是没有错误的。但是推出一个没有错误的软件几乎不可能，所以很多软件公司都在疯狂地发放补丁（有时这种补丁甚至是收费的）。T公司就是其中之一。上个月，T公司推出了一个新的字处理软件，随后发放了一批补丁。最近T公司发现其发放的补丁有致命的问题，那就是一个补丁在排除某些错误的同时，往往会加入另一些错误，此字处理软件中只可能出现 n 个特定的错误，这 n 个错误是由软件本身决定的。T公司目前共发放了 m 个补丁，**对于每一个补丁，都有特定的适用环境，某个补丁只有在当前软件中包含某些错误而同时又不包含另一些错误时才可以使用，如果它被使用，它将修复某些错误而同时加入某些错误。**另外，使用每个补丁都要耗一定的时间（即补丁程序的运行时间）。

现在T公司的问题很简单：其字处理软件的初始版本不幸地包含了全部 n 个错误，有没有可能通过使用这 m 个补丁（任意顺序地使用，一个补丁可使用多次），使此字处理软件成为一个没有错误的软件。如果可能，希望找到总耗时最少的方案。

计算机问题求解实例（续）

任务描述：T公司的字处理软件中可能出现的n个错误为集合 $B = \{b1, b2, \dots, bn\}$ 中的元素，T公司目前共发放了m个补丁，记为； $P = \{p1, p2, \dots, pm\}$ 。对于每一个补丁 P_i ，假设存在错误集合： B_{i+}, B_{i-} ，当软件包含了 B_{i+} 中的所有错误，而没有包含 B_{i-} 中的任何错误时，补丁 P_i 才可以被使用，否则不能使用，显然 B_{i+}, B_{i-} 交集为空。补丁 P_i 将修复某些错误而同时加入某些错误，设错误集合 F_{i+}, F_{i-} 使用过补丁 P_i 之后， F_{i-} 中的任何错误都不会在软件中出现，而软件将包含 F_{i+} 中的所有错误，同样 F_{i+}, F_{i-} 交集为空。另外，使用每个补丁都要耗一定的时间 T_i 。

现在T公司的字处理软件的初始版本不幸地包含了全部n个错误，试编写程序判断有没有可能通过使用这m个补丁（任意顺序地使用，一个补丁可使用多次），使此字处理软件成为一个没有错误的软件。如果可能，希望找到总耗时最少的方案。

计算机问题求解实例（续）

□ 构建模型

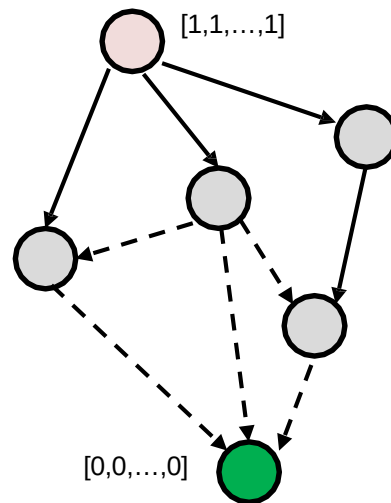
把 n 个bug的状态（存在和不存在）的组合用一个0-1字符串（模式串）表示，执行一个补丁本质上就是模式串的转变。模式串做为一个图的顶点，补丁则是边，问题则转换为从全1模式串到全0模式串的转变。

□ 设计最短路径算法

Dijkstra最短路径算法求解

□ 编程实现

□ 测试



第1章 基础知识

1.1 有关算法的基本概念

几个例子

问题、算法、算法的时间复杂度

1.2 算法的伪码表示

1.3 数学基础知识

1.3.1 函数的渐近的界

1.3.2 求和的方法

1.3.3 递推方程求解方法

1.1 有关算法的基本概念

几个算法例子

例1 调度问题

已知：任务集 $S=\{1, 2, \dots, n\}$,

第 j 项任务加工时间: $t_j, \in \mathbb{Z}^+, j=1, 2, \dots, n$

一个可行调度方案: $1, 2, \dots, n$ 的排列 $i_1, i_2, \dots, i_n$, 使每项任务的等待时间之和最少。

实例

任务集 $S = \{1, 2, 3, 4, 5\}$,

加工时间: $t_1=3, t_2=8, t_3=5, t_4=10, t_5=15$

问题建模

输入：任务集： $S=\{1, 2, \dots, n\}$,

第 j 项任务加工时间： $t_j \in \mathbb{Z}^+$, $j=1, 2, \dots, n$.

输出：调度 I , S 的排列 $i_1, i_2, \dots, i_n$,

目标函数： I 的完成时间, $t(I) = \sum_{k=1}^n (n - k + 1)t_{i_k}$

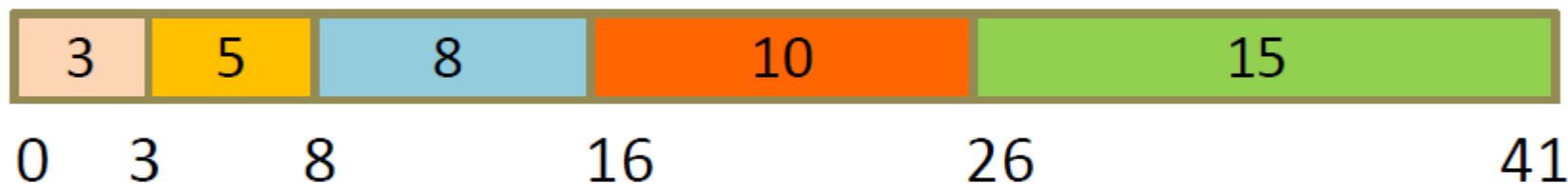
解 I^* : 使得 $t(I^*)$ 达到最小, 即

$$t(I^*) = \min \{ t(I) \mid I \text{ 为 } S \text{ 的排列} \}$$

贪心法的解

算法：按加工时间 (3,8,5,10,15) 从小到大安排

解：1, 3, 2, 4, 5



总完成时间：

$$\begin{aligned}t &= 3 + (3+5) + (3+5+8) + (3+5+8+10) + (3+5+8+10+15) \\&= 3 \times 5 + 5 \times 4 + 8 \times 3 + 10 \times 2 + 15 \\&= 94\end{aligned}$$

贪心算法

设计策略：加工时间短的先做

算法：根据加工时间从小到大排序,依次加工

算法正确性：对所有输入实例都得到最优解

证：假如调度 f 第 i, j 项任务相邻且有逆序，
即 $t_i > t_j$ 。交换任务 i 和 j 得到调度 g ，



总完成时间 $t(g) - t(f) = t_j - t_i < 0$

直觉不一定是正确的

反例

有4件物品要装入背包, 物品重量和价值如下:

标号	1	2	3	4
重量 w_i	3	4	5	2
价值 v_i	7	9	9	2

背包重量限制是 6, 问如何选择物品, 使得不超重的情况下装入背包的物品价值达到最大?

0-1背包问题建模

问题的解：0-1向量 $\langle x_1, x_2, \dots, x_n \rangle$
 $x_i = 1 \Leftrightarrow$ 物品 i 装入背包

目标函数 $\max \sum_{i=1}^n v_i x_i$

约束条件 $\sum_{i=1}^n w_i x_i \leq B$

$$x_i = 0, 1, i = 1, 2, \dots, n$$

实例的解

贪心法： 单位重量价值大的优先，总重不超 6

按照 $\frac{v_i}{w_i}$ 从大到小排序：1, 2, 3, 4

$$\frac{7}{3} > \frac{9}{4} > \frac{9}{5} > \frac{2}{2}$$

贪心法的解：{ 1, 4 }，重量 5，价值为 9.

更好的解：{ 2, 4 }，重量 6，价值 11.

算法设计

1. 问题建模
2. 选择什么算法？如何描述这个方法？
3. 这个方法是否对所有实例都得到最优解？
如何证明？
4. 如果不是，能否找到反例？

例2：投资问题

问题： m 元钱，投资 n 个项目. 效益函数 $f_i(x)$, 表示第 i 个项目投 x 元的效益, $i=1, 2, \dots, n$. 求如何分配每个项目的钱数使得总效益最大?

实例： 5 万元，投资给 4 个项目，效益函数：

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

建模

输入: $n, m, f_i(x), i=1,2, \dots, n, x=1,2, \dots, m$

解: n 维向量 $\langle x_1, x_2, \dots, x_n \rangle$, x_i 是第 i 个项目的钱数, 使得下述条件满足:

目标函数

$$\max \sum_{i=1}^n f_i(x_i),$$

约束条件

$$\sum_{i=1}^n x_i = m, \quad x_i \in \mathbb{N}$$

蛮力算法

对所有满足下述条件的向量 $\langle x_1, x_2, \dots, x_n \rangle$

$$x_1 + x_2 + \dots + x_n = m$$

x_i 为非负整数, $i = 1, 2, \dots, n$

计算相应的效益

$$f_1(x_1) + f_2(x_2) + \dots + f_n(x_n)$$

从中确认效益最大的向量.

实例计算

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

$$x_1 + x_2 + x_3 + x_4 = 5$$

$$s_1 = \langle 0, 0, 0, 5 \rangle, v(s_1) = 24$$

$$s_2 = \langle 0, 0, 1, 4 \rangle, v(s_2) = 25$$

$$s_3 = \langle 0, 0, 2, 3 \rangle, v(s_3) = 32$$

...

$$s_{56} = \langle 5, 0, 0, 0 \rangle, v(s_{56}) = 15$$

解: $s = \langle 1, 0, 3, 1 \rangle$,

最大效益: $11 + 30 + 20 = 61$

蛮力算法的效率

方程 $x_1 + x_2 + \dots + x_n = m$ 的非负整数解
 $\langle x_1, x_2, \dots, x_n \rangle$ 的个数估计:

可行解表示成 0-1 序列: m 个1, $n-1$ 个 0

$\underbrace{1 \dots 1}_{x_1 \uparrow} 0 \underbrace{1 \dots 1}_{x_2 \uparrow} 0 \dots 0 \underbrace{1 \dots 1}_{x_n \uparrow}$

$n=4, m=7$

可行解 $\langle 1, 2, 3, 1 \rangle$

$\Leftrightarrow$ 序列 $1 0 1 1 0 1 1 1 0 1$

蛮力算法的效率

序列个数是输入规模的指数函数

$$\begin{aligned} & C(m+n-1, m) \\ &= \frac{(m+n-1)!}{m!(n-1)!} \\ &= \Omega((1+\varepsilon)^{m+n-1}) \end{aligned}$$



有没有更好的算法？

例3 排序算法的评价

例3 排序算法

考察元素比较次数

- 插入排序、冒泡排序：最坏和平均状况下都为 $O(n^2)$
- 快速排序：最坏状况为 $O(n^2)$ ，平均状况下为 $O(n\log n)$
- 堆排序、二分归并排序：最坏和平均状况下都为 $O(n\log n)$
- ...

问题

- 那个排序算法效率最高？
- 是否可以找到更好的算法？排序问题的计算难度如何估计？

货郎问题

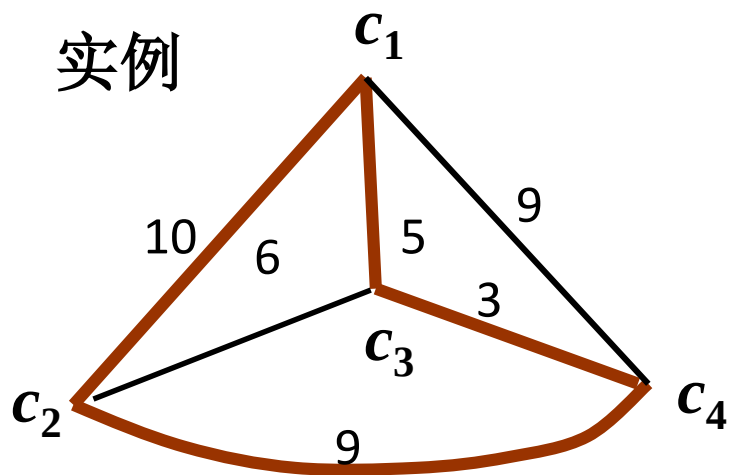
例4 货郎问题:

- 有穷个城市的集合 $C = \{c_1, c_2, \dots, c_m\}$, 距离 $d(c_i, c_j) = d(c_j, c_i) \in \mathbb{Z}^+$, $1 \leq i < j \leq m$

- 求 $1, 2, \dots, m$ 的排列 $k_1, k_2, \dots, k_m$ 使得

$$\min \left\{ \sum_{i=1}^{m-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_m}, c_{k_1}) \right\}$$

实例



现状: 至今没有找到有效的算法,
存在大量问题与它难度等价

问题: 是否存在有效算法?

如何处理这类问题?

双机调度

双机调度问题：有 n 项任务，任务 i 的加工时间为 t_i , $t_i \in \mathbb{Z}^+$, $i=1,2,\dots,n$. 用两台相同的机器加工，从0时刻开始计时，完成时间是后停止加工机器的停机时间. 问如何把这些任务分配到两台机器上，使得完成时间达到最小？

实例：任务集 $S = \{1,2,3,4,5,6\}$

$$\underline{t_1=3}, \underline{t_2=10}, t_3=6, \underline{t_4=2}, t_5=1, t_6=7$$

解：机器 1 的任务：1, 2, 4

机器 2 的任务：3, 5, 6

完成时间： $\max\{3+10+2, 6+1+7\}=15$

双机调度建模

解: 0-1向量 $\langle x_1, x_2, \dots, x_n \rangle$, $x_i=1$ 表示任务 i 分配到第一台机器, $i=1,2,\dots,n$.

不妨设机器1的加工时间 $\leq$ 机器2的加工时间令 $T=t_1+t_2+\dots+t_n$, $D=\lfloor T/2 \rfloor$, 机器1的加工时间不超过 D , 且达到最大.



如何对该问题建模? 目标函数与约束条件是什么?

NP-hard问题

- 这样的问题有数千个，大量存在于各个应用领域.
- 至今没找到有效算法：现有的算法的运行时间是输入规模的指数或更高阶函数.
- 至今没有人能够证明对于这类问题不存在多项式时间的算法.
- 从是否存在多项式时间算法的角度看，这些问题彼此是等价的. 这些问题的难度处于可有效计算的边界.

什么是好的算法？

- **空间复杂度 $S(n)$** —— 根据算法写成的程序在执行时**占用存储单元的长度**。这个长度往往与输入数据的规模有关。空间复杂度过高的算法可能导致使用的内存超限，造成程序非正常中断。
 - **时间复杂度 $T(n)$** —— 根据算法写成的程序在执行时**耗费时间的长度**。这个长度往往也与输入数据的规模有关。时间复杂度过高的低效算法可能导致我们在有生之年都等不到运行结果。
-

算法分析的原则

- 机器的运算速度影响算法的运行时间

机器	运算速度	运行算法	运行时间
天河三号	百亿亿次/秒	插入排序	无法公平比较
个人电脑	十亿次/秒	选择排序	



分析算法的运行时间应独立于机器

算法分析的原则

- 归纳基本操作
 - 如：运算、赋值、比较
- 统一机器性能
 - 假设基本操作代价均为**1**

+	-	×	÷
:=	>	<	=



统一机器性能后，算法运行时间依赖于问题输入规模与实例

基本运算与输入规模

- **算法时间复杂度**: 针对指定基本运算, 计数算法所做运算次数
- **基本运算**: 比较, 加法, 乘法, 置指针, 交换...
- **输入规模**: 输入串编码长度
通常用下述参数度量: 数组元素多少, 调度问题的任务个数, 图的顶点数与边数等.
- **算法基本运算次数可表为输入规模的函数**
- **给定问题和基本运算就决定了一个算法类**

输入规模

- 排序：数组中元素个数 n
- 检索：被检索数组的元素个数 n
- 整数乘法：两个整数的位数 m, n
- 矩阵相乘：矩阵的行列数 i, j, k
- 图的遍历：图的顶点数 n , 边数 m
- ...

基本运算

- 排序: 元素之间的比较
- 检索: 被检索元素 x 与数组元素的比较
- 整数乘法: 每位数字相乘(位乘) 1 次
 m 位和 n 位整数相乘要做 mn 次位乘
- 矩阵相乘: 每对元素乘 1 次
 $i \times j$ 矩阵与 $j \times k$ 矩阵相乘要做 ijk 次乘法
- 图的遍历: 置指针

算法的两种时间复杂度

对于相同输入规模的不同实例，算法的基本运算次数也不一样，可定义两种时间复杂度

最坏情况下的时间复杂度 $W(n)$

算法求解输入规模为 n 的实例所需要的最长时间

平均情况下的时间复杂度 $A(n)$

在给定同样规模为 n 的输入实例的概率分布下，算法求解这些实例所需要的平均时间

$A(n)$ 计算公式

平均情况下的时间复杂度 $A(n)$

设 S 是规模为 n 的实例集

实例 $I \in S$ 的概率是 P_I

算法对实例 I 执行的基本运算次数是 t_I

$$A(n) = \sum_{I \in S} P_I t_I$$

在某些情况下可以假定每个输入实例概率相等

例子：检索

检索问题

输入：非降顺序排列的数组 L , 元素数 n ,
数 x

输出： j

若 x 在 L 中, j 是 x 首次出现的下标;
否则 $j = 0$

基本运算： x 与 L 中元素的比较

顺序检索算法

$j=1$, 将 x 与 $L[j]$ 比较. 如果 $x=L[j]$, 则算法停止, 输出 j ; 如果不等, 则把 j 加1, 继续 x 与 $L[j]$ 的比较, 如果 $j>n$, 则停机并输出0.

实例

1	2	3	4	5
---	---	---	---	---

$x = 4$, 需要比较 4 次

$x = 2.5$, 需要比较 5 次

最坏情况的时间估计

不同的输入有 $2n + 1$ 个，分别对应：

$$x = L[1], x = L[2], \dots, x = L[n]$$

$$x < L[1], L[1] < x < L[2], L[2] < x < L[3], \dots, L[n] < x$$

最坏情况下时间： $W(n) = n$

最坏的输入： x 不在 L 中或 $x = L[n]$

要做 n 次比较

平均情况的时间估计

输入实例的概率分布:

假设 x 在 L 中概率是 p ,且每个位置概率相等

$$A(n) = \sum_{i=1}^n i \frac{p}{n} + (1-p)n$$

等差数列求和

$$= \frac{p(n+1)}{2} + (1-p)n$$

当 $p=1/2$ 时,

$$A(n) = \frac{n+1}{4} + \frac{n}{2} \approx \underline{\frac{3n}{4}}$$

实例：改进的顺序检索

Search(L, x)

输入：数组 $L[1..n]$ ，其元素按照从小到大排列，数 x .

输出：若 x 在 L 中，输出 x 的位置下标 j ；否则输出0.

1. $j \leftarrow 1$
2. while $j \leq n$ and $x > L[j]$ do $j \leftarrow j + 1$
3. if $x < L[j]$ or $j > n$ then $j \leftarrow 0$
4. return j

改进顺序检索算法

$j=1$, 将 x 与 $L[j]$ 比较. 如果 $x=L[j]$, 则算法停止, 输出 j ; 如果 $x > L[j]$, 则把 j 加1, 继续 x 与 $L[j]$ 的比较; 如果 $x < L[j]$, 则停机并输出0. 如果 $j > n$, 则停机并输出 0.

实例

1	2	3	4	5
---	---	---	---	---

$x = 4$, 需要比较 4 次

$x = 2.5$, 需要比较 3 次

时间估计

最坏情况下: $W(n) = n$

平均情况下

输入实例的概率分布: 假设 x 在 L 中每个位置与空隙的概率都相等



改进检索算法平均时间复杂度是多少?

小结:

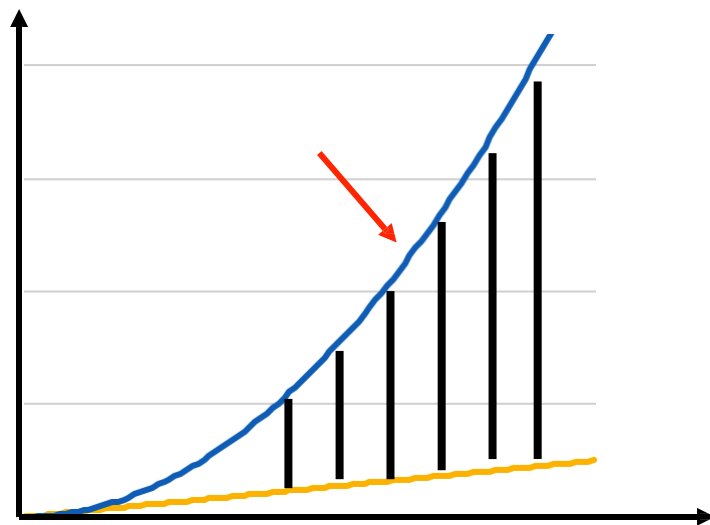
- 算法最坏和平均情况下的时间复杂度定义
- 如何计算上述时间复杂度

算法分析的原则

- 相同输入规模，实例影响运行

输入情况	情况说明
最好情况	不常出现，不具普遍性
最坏情况	确定上界，更具一般性
一般情况	情况复杂，分析难度大

运行时间



常用最坏情况分析算法运行时间

最好情况

n

算法运行时间仅依赖于问题输入规模 n ，表示为 $T(n)$

算法分析的工具

- 插入排序最坏情况

```

for j ← 2 to n do -----  $n$ 次
    key ← A[j] -----  $n-1$ 次
    i ← j - 1 -----  $n-1$ 次
    while i > 0 and A[i] > key do -----  $\sigma_{k=2}^n k$ 次
        | A[i+1] ← A[i] -----  $\sigma_{k=2}^n k-1$ 次
        | i ← i - 1 -----  $\sigma_{k=2}^n k-1$ 次
    end
    A[i+1] ← key -----  $n-1$ 次
end
    
```

- 选择排序最坏情况

```

for i ← 1 to n - 1 do -----  $n$ 次
    cur_min ← A[i] -----  $n-1$ 次
    cur_min_pos ← i -----  $n-1$ 次
    for j ← i + 1 to n do -----  $\sigma_{k=0}^{n-2}(n-k)$ 次
        if A[j] < cur_min_pos then -----  $\sigma_{k=1}^{n-1}(n-k)$ 次
            | cur_min ← A[j] -----  $\sigma_{k=1}^{n-1}(n-k)$ 次
            | cur_min_pos ← j -----  $\sigma_{k=1}^{n-1}(n-k)$ 次
        end
    end
    交换 A[i] 和 A[cur_min_pos] -----  $\sigma_{k=1}^{n-1}(n-k)$ 次
end -----  $n-1$ 次
    
```

计算每行伪代码操作次数

算法分析的工具

- 插入排序最坏情况

```

for j ← 2 to n do
    key ← A[j]
    i ← j - 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i]
        i ← i - 1
    end
    A[i + 1] ← key
end
    
```

代码复杂度分析 (次):
 for j ← 2 to n do: n 次
 key ← A[j]: $n - 1$ 次
 i ← j - 1: $n - 1$ 次
 while i > 0 and A[i] > key do: $\sum_{k=2}^n k$ 次
 A[i + 1] ← A[i]: $\sum_{k=2}^n k - 1$ 次
 i ← i - 1: $\sum_{k=2}^n k - 1$ 次
 end: $n - 1$ 次
 A[i + 1] ← key: $n - 1$ 次
 end: $n - 1$ 次

求和

$$T(n) = \frac{3}{2}n^2 + \frac{7}{2}n - 4$$

- 选择排序最坏情况

```

for i ← 1 to n - 1 do
    cur_min ← A[i]
    cur_min_pos ← i
    for j ← i + 1 to n do
        if A[j] < cur_min then
            cur_min ← A[j]
            cur_min_pos ← j
        end
    end
    交换 A[i] 和 A[cur_min_pos]
end
    
```

代码复杂度分析 (次):
 for i ← 1 to n - 1 do: n 次
 cur_min ← A[i]: $n - 1$ 次
 cur_min_pos ← i: $n - 1$ 次
 for j ← i + 1 to n do: $\sum_{k=0}^{n-2} (n - k)$ 次
 if A[j] < cur_min then: $\sum_{k=1}^{n-1} (n - k)$ 次
 cur_min ← A[j]: $\sum_{k=1}^{n-1} (n - k)$ 次
 cur_min_pos ← j: $\sum_{k=1}^{n-1} (n - k)$ 次
 end: $n - 1$ 次
 交换 A[i] 和 A[cur_min_pos]: $n - 1$ 次
 end: $n - 1$ 次

求和

$$T(n) = 2n^2 + 3n - 4$$

算法分析的工具

插入排序最坏情况

```

for j ← 2 to n do
    key ← A[j]
    i ← j - 1
    while i > 0 and A[i] > key do
        A[i + 1] ← A[i]
        i ← i - 1
    end
    A[i + 1] ← key
end
    
```

代码右侧标注的复杂度次数：
 for j ← 2 to n do: n 次
 key ← A[j]: $n - 1$ 次
 i ← j - 1: $n - 1$ 次
 while i > 0 and A[i] > key do: $\sigma_{k=2}^n k$ 次
 A[i + 1] ← A[i]: $\sigma_{k=2}^n k - 1$ 次
 i ← i - 1: $\sigma_{k=2}^n k - 1$ 次
 end: $n - 1$ 次
 A[i + 1] ← key: $n - 1$ 次
 end: $n - 1$ 次

求和

$$T(n) = \frac{3}{2}n^2 + \frac{7}{2}n - 4$$

选择排序最坏情况

```

for i ← 1 to n - 1 do
    cur_min ← A[i]
    cur_min_pos ← i
    for j ← i + 1 to n do
        if A[j] < cur_min_pos then
            cur_min ← A[j]
            cur_min_pos ← j
        end
    end
    交换 A[i] 和 A[cur_min_pos]
end
    
```

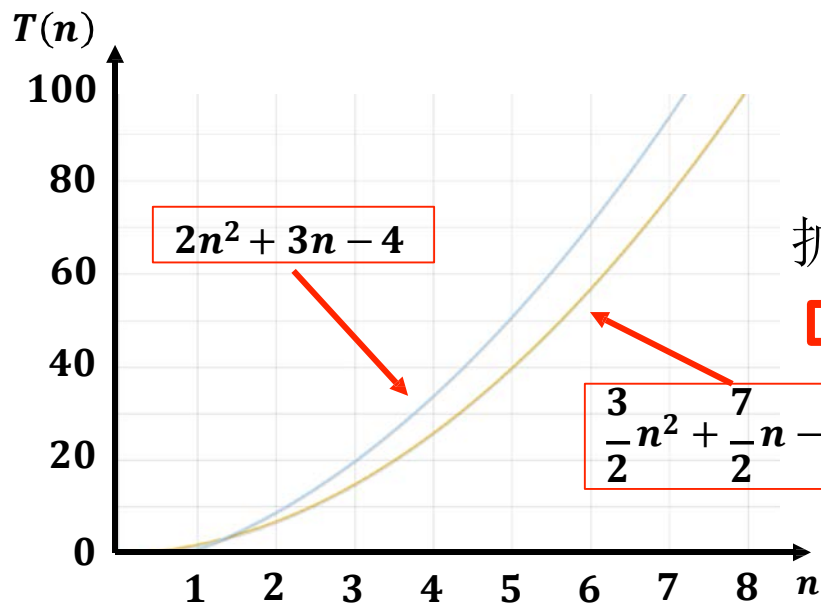
代码右侧标注的复杂度次数：
 for i ← 1 to n - 1 do: n 次
 cur_min ← A[i]: $n - 1$ 次
 cur_min_pos ← i: $n - 1$ 次
 for j ← i + 1 to n do: $\sigma_{k=0}^{n-2} (n - k)$ 次
 if A[j] < cur_min_pos then: $\sigma_{k=1}^{n-1} (n - k)$ 次
 cur_min ← A[j]: $\sigma_{k=1}^{n-1} (n - k)$ 次
 cur_min_pos ← j: $\sigma_{k=1}^{n-1} (n - k)$ 次
 end: $n - 1$ 次
 交换 A[i] 和 A[cur_min_pos]: $n - 1$ 次
 end: $n - 1$ 次

求和

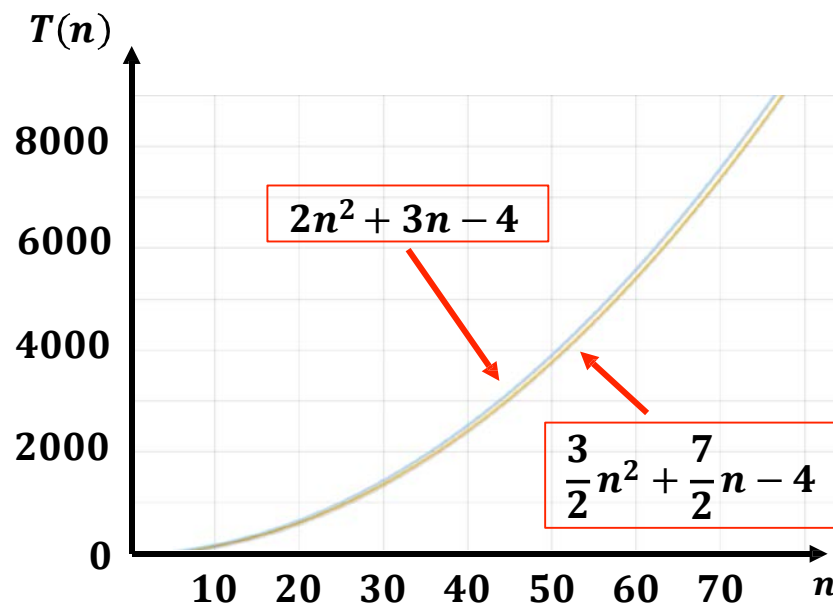
$$T(n) = 2n^2 + 3n - 4$$

问题：能否简洁地衡量算法运行时间？

算法分析的工具



扩展规模



- 在 n 充分大时，两者相差不大
- 原因：两函数的最高阶项相同

算法分析的工具

- 插入排序最坏情况

```
for  $j \leftarrow 2$  to  $n$  do
     $key \leftarrow A[j]$ 
     $i \leftarrow j - 1$ 
    while  $i > 0$  and  $A[i] > key$  do
         $A[i + 1] \leftarrow A[i]$ 
         $i \leftarrow i - 1$ 
    end
     $A[i + 1] \leftarrow key$ 
end
```

$$T(n) = \frac{3}{2}n^2 + \frac{7}{2}n - 4$$

- 选择排序最坏情况

```
for  $i \leftarrow 1$  to  $n - 1$  do
     $cur\_min \leftarrow A[i]$ 
     $cur\_min\_pos \leftarrow i$ 
    for  $j \leftarrow i + 1$  to  $n$  do
        if  $A[j] < cur\_min$  then
             $cur\_min \leftarrow A[j]$ 
             $cur\_min\_pos \leftarrow j$ 
        end
    end
    交换  $A[i]$  和  $A[cur\_min\_pos]$ 
end
```

$$T(n) = 2n^2 + 3n - 4$$

渐近分析：忽略 $T(n)$ 的系数与低阶项，仅关注高阶项，用记号 Θ 表示

算法分析的工具

- 插入排序最坏情况

```
for  $j \leftarrow 2$  to  $n$  do
   $key \leftarrow A[j]$ 
   $i \leftarrow j - 1$ 
  while  $i > 0$  and  $A[i] > key$  do
     $A[i + 1] \leftarrow A[i]$ 
     $i \leftarrow i - 1$ 
  end
   $A[i + 1] \leftarrow key$ 
end
```

$$T(n) = \Theta(n^2)$$

- 选择排序最坏情况

```
for  $i \leftarrow 1$  to  $n - 1$  do
   $cur\_min \leftarrow A[i]$ 
   $cur\_min\_pos \leftarrow i$ 
  for  $j \leftarrow i + 1$  to  $n$  do
    if  $A[j] < cur\_min$  then
       $cur\_min \leftarrow A[j]$ 
       $cur\_min\_pos \leftarrow j$ 
    end
  end
  交换  $A[i]$  和  $A[cur\_min\_pos]$ 
end
```

$$T(n) = \Theta(n^2)$$

渐近分析：忽略 $T(n)$ 的系数与低阶项，仅关注高阶项，用记号 Θ 表示

时间复杂度表示 — 函数简化

对于算法A的复杂度函数 $T(N)$ ，如果存在 $T'(N)$ ，使得当 $N \rightarrow \infty$ 时有 $(T(N) - T'(N))/T(N) \rightarrow 0$ ，那么就称 $T'(N)$ 为算法A当 $N \rightarrow \infty$ 的渐近复杂性

例1: $T(N) = 10N^2 + 100N + 10000$

$$T'(N) = 10N^2$$

例2: $T(N) = 2^N + \sigma_{i=1}^{100} i * N^i + 10000000000000$

$$T'(N) = 2^N$$

- $T'(N)$ 是 $T(N)$ 略去低阶项所留下的主项，表达简单
- 如果两个算法的渐近复杂性的阶不相同，那么只要确定出各自的阶就可以判断哪一个算法效率高
- 阶跟 $T'(N)$ 中的常数因子没有关系， $T'(N)$ 可进一步简化，省略常数因子

算法的表示

- 伪代码

- 非正式语言

- 移植**编程语言**书写形式作为基础和框架
 - 按照接近**自然语言**的形式表达算法过程

- 兼顾自然语言与编程语言优势

- **简洁**表达算法本质，不拘泥于实现细节
 - **准确**反映算法过程，不产生矛盾和歧义

选择排序

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$ 
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$ 
for  $i \leftarrow 1$  to  $n - 1$  do
     $cur\_min \leftarrow A[i]$ 
     $cur\_min\_pos \leftarrow i$ 
    for  $j \leftarrow i + 1$  to  $n$  do
        if  $A[j] < cur\_min$  then
            //记录当前最小值及其位置
             $cur\_min \leftarrow A[j]$ 
             $cur\_min\_pos \leftarrow j$ 
        end
    end
    交换  $A[i]$  和  $A[cur\_min\_pos]$ 
end
```

算法的表示方式比较

表示方式	语言特点
自然语言	贴近人类思维，易于理解主旨 表述不够精准，存在模糊歧义
编程语言	精准表达逻辑，规避表述歧义 受限语法细节，增大理解难度
伪代码	关注算法本质，便于书写阅读

算法的伪码描述

- 赋值语句: $\leftarrow$
- 分支语句: if ...then ... [else...]
- 循环语句: while, for, repeat until
- 转向语句: goto
- 输出语句: return
- 调用: 直接写过程的名字
- 注释: //...

实例：求最大公约数

算法1.1 Euclid(m, n)

输入：非负整数 m, n ，其中 m 与 n 不全为 0

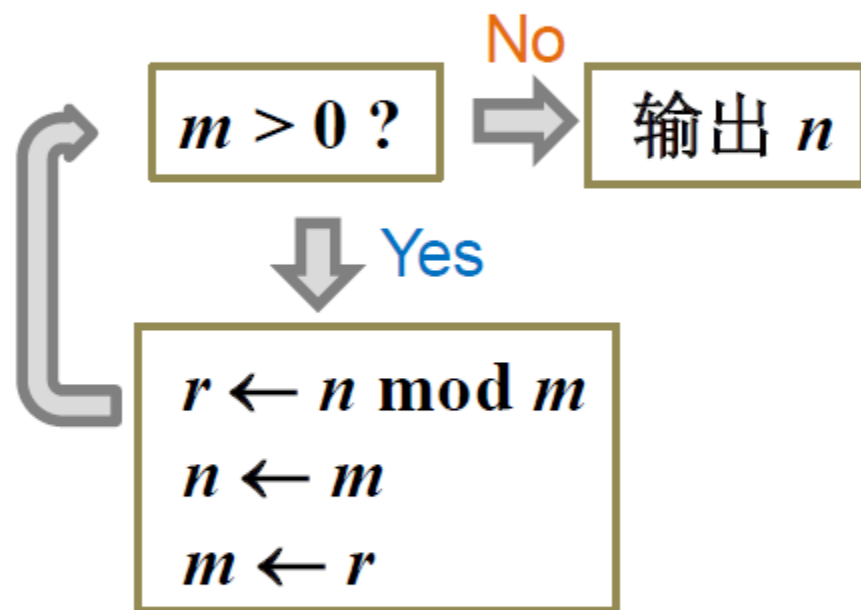
输出： m 与 n 的最大公约数

1. while $m > 0$ do
2. $r \leftarrow n \bmod m$
3. $n \leftarrow m$
4. $m \leftarrow r$
5. return n

运行实例: $n=36, m=15$

while	n	m	r
第1次	36	15	6
第2次	15	6	3
第3次	6	3	0
	3	0	0

↓
输出3



例：插入排序

算法 **Insert Sort** (A, n)

输入： n 个数的数组 A

输出： 按照递增顺序排好序的数组 A

1. for $j \leftarrow 2$ to n do
2. $x \leftarrow A[j]$
3. $i \leftarrow j-1$ //3-7 行把 $A[j]$ 插入 $A[1..j-1]$
4. while $i > 0$ and $x < A[i]$ do
5. $A[i+1] \leftarrow A[i]$
6. $i \leftarrow i-1$
7. $A[i+1] \leftarrow x$

运行实例

2	4	1	5	3
---	---	---	---	---

$j = 3, x = A[3] = 1$

$i = 2, A[2] = 4$

$i > 0, x < A[2] \quad \checkmark$

2	4	4	5	3
---	---	---	---	---

$A[3] = 4, i = 1, x = 1$

$i > 0, x < A[1] \quad \checkmark$

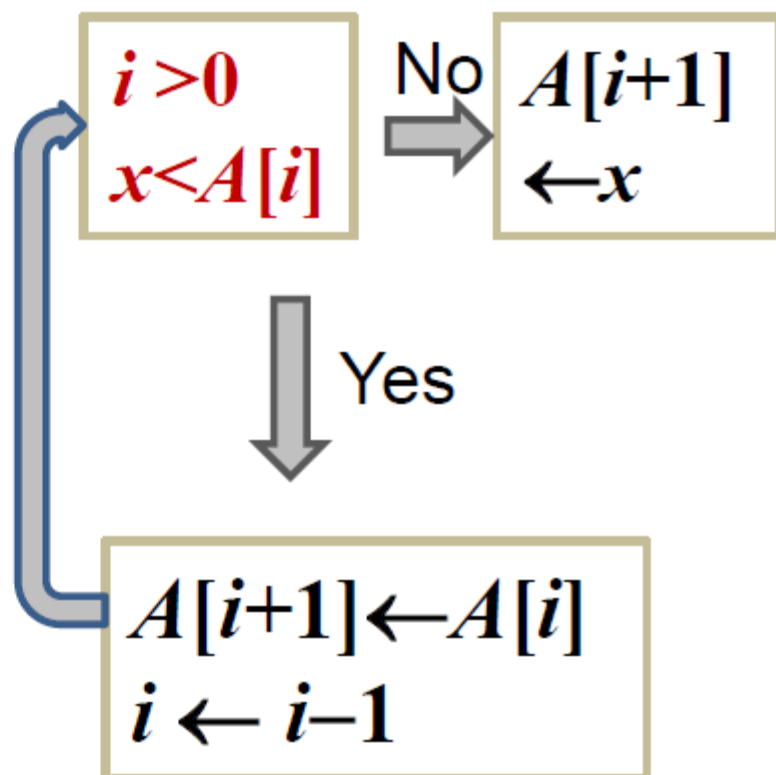
2	2	4	5	3
---	---	---	---	---

$A[2] = 2, i = 0, x = 1$

$i > 0 \quad \times$

1	2	4	5	3
---	---	---	---	---

4. while $i > 0$ and $x < A[i]$ do
5. $A[i+1] \leftarrow A[i]$
6. $i \leftarrow i-1$
7. $A[i+1] \leftarrow x$



例：二分归并排序

MergeSort (A, p, r)

输入：数组 $A[p..r]$

输出：按递增顺序排序的数组 A

1. if $p < r$
2. then $q \leftarrow \lfloor (p+r)/2 \rfloor$
3. MergeSort (A, p, q)
4. MergeSort ($A, q+1, r$)
5. Merge (A, p, q, r)

MergeSort有递归调用,也调用Merge过程

例：算法A的伪码

算法 A

输入：实数的数组 $P[0..n]$ ，实数 x

输出： y

1. $y \leftarrow P[0]; \text{ power} \leftarrow 1$
2. for $i \leftarrow 1$ to n do
3. $\text{power} \leftarrow \text{power} * x$
4. $y \leftarrow y + P[i] * \text{power}$
5. return y

算法
A计
算什么值
？

对 $i = 1, 2, \dots, n$ $\Rightarrow$

$power \leftarrow power * x$
 $y \leftarrow y + P[i] * power$

	i	$power$	y
初值		1	$P[0]$
循环	1	x	$P[0] + P[1]*x$
	2	x^2	$P[0] + P[1]*x + P[2]*x^2$
	3	x^3	$P[0] + P[1]*x + P[2]*x^2 + P[3]*x^3$
			...

输入 $P[0..n]$ 是 n 次多项式 $P(x)$ 的系数
算法 A 计算该多项式在 x 的值

1.3 数学基础：函数的渐近的界

定义1.1 设 f 和 g 是定义域为自然数集 $\mathbb{N}$ 上的函数.

- (1) 若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$ 成立, 则称 $f(n)$ 的渐近的上界是 $g(n)$, 记作 $f(n) = O(g(n))$.
- (2) 若存在正数 c 和 n_0 , 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$ 成立, 则称 $f(n)$ 的渐近的下界是 $g(n)$, 记作 $f(n) = \Omega(g(n))$.
- (3) 若对任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq f(n) < cg(n)$ 成立, 则记作 $f(n) = o(g(n))$.
- (4) 若对于任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq cg(n) < f(n)$ 成立, 则记作 $f(n) = \omega(g(n))$.
- (5) 若 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$, 则记作 $f(n) = \Theta(g(n))$.

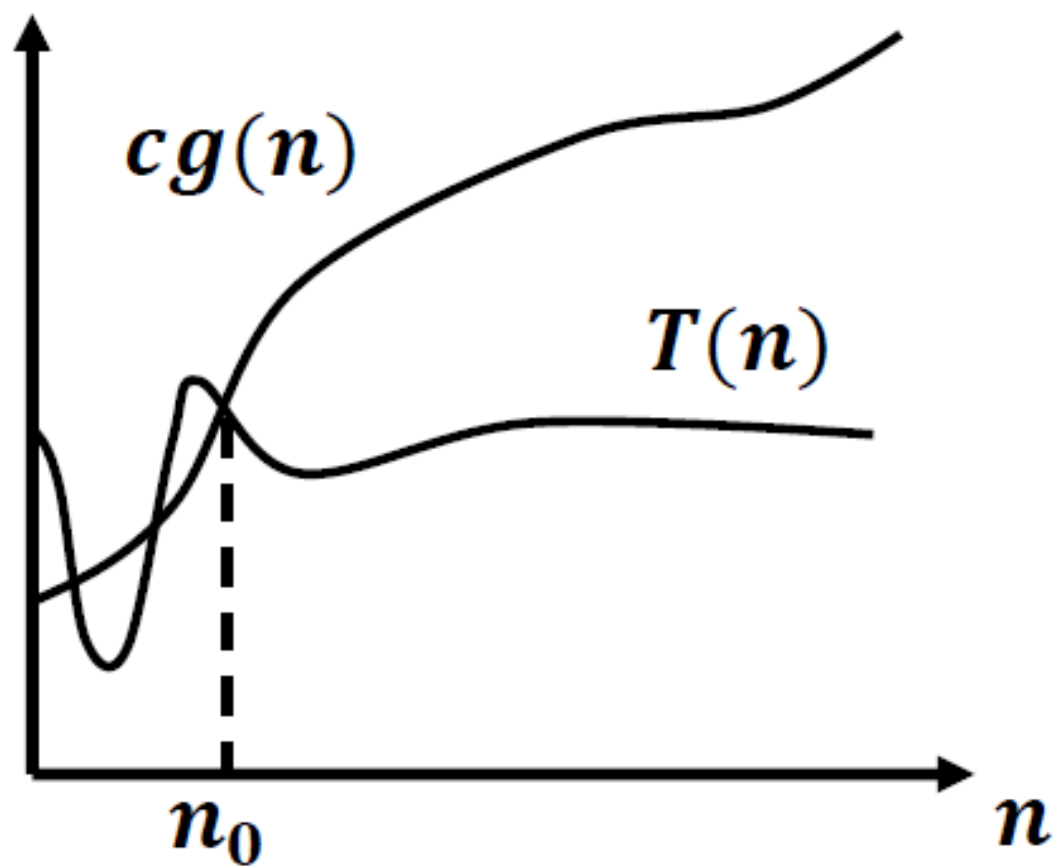
例子

设 $f(n) = n^2 + n$, 则

$f(n) = O(n^2)$, 取 $c = 2$, $n_0 = 1$ 即可

$f(n) = O(n^3)$, 取 $c = 1$, $n_0 = 2$ 即可

1. $f(n) = O(g(n))$, $f(n)$ 的阶不高于 $g(n)$ 的阶.
2. 可能存在多个正数 c , 只要指出一个即可.
3. 对前面有限个值可以不满足不等式.
4. 常函数可以写作 $O(1)$.



$$T(n) = O(g(n))$$

运算法则

根据 O 的定义，容易证明它有如下运算规则：

- 1) $O(f) + O(g) = O(\max(f, g))$;
- 2) **$O(f) + O(g) = O(f + g)$** ;
- 3) $O(f) O(g) = O(fg)$;
- 4) 如果 $g(N) = O(f(N))$ ，则 $O(f) + O(g) = O(f)$;
- 5) $O(Cf(N)) = O(f(N))$ ，其中 C 是一个正的常数；
- 6) $f = O(f)$ 。

定理3

定理 假设函数 f 和 g 的定义域为自然数集, 若对某个其它函数 h , 有 $f=O(h)$ 和 $g=O(h)$, 那么

$$f + g = O(h).$$

该性质可以推广到有限个函数.

算法由有限步骤构成. 若每一步的时间复杂度函数的上界都是 $h(n)$, 那么该算法的时间复杂度函数可以写作 $O(h(n))$.

试证明： $O(f) + O(g) = O(f + g)$

证明：

$F = O(f)$, 存在 n_1 和 c_1 , 对于任意 n , 当 $n \geq n_1$ 时满足: $F(n) \leq c_1 f(n)$

$G = O(g)$, 存在 n_2 和 c_2 , 对于任意 n , 当 $n \geq n_2$ 时满足: $G(n) \leq c_2 g(n)$

取 $n_3 = \max(n_1, n_2)$, $c_3 = \max(c_1, c_2)$, 当 $n \geq n_3$ 时满足:

$$O(f) + O(g) = F(n) + G(n) \leq c_3(f(n) + g(n))$$

因此, $O(f) + O(g) = O(f + g)$

1.3 数学基础：函数的渐近的界

定义1.1 设 f 和 g 是定义域为自然数集 N 上的函数.

- (1) 若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$ 成立, 则称 $f(n)$ 的渐近的上界是 $g(n)$, 记作 $f(n) = O(g(n))$.
- (2) 若存在正数 c 和 n_0 , 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$ 成立, 则称 $f(n)$ 的渐近的下界是 $g(n)$, 记作 $f(n) = \Omega(g(n))$.
- (3) 若对任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq f(n) < cg(n)$ 成立, 则记作 $f(n) = o(g(n))$.
- (4) 若对于任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq cg(n) < f(n)$ 成立, 则记作 $f(n) = \omega(g(n))$.
- (5) 若 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$, 则记作 $f(n) = \Theta(g(n))$.

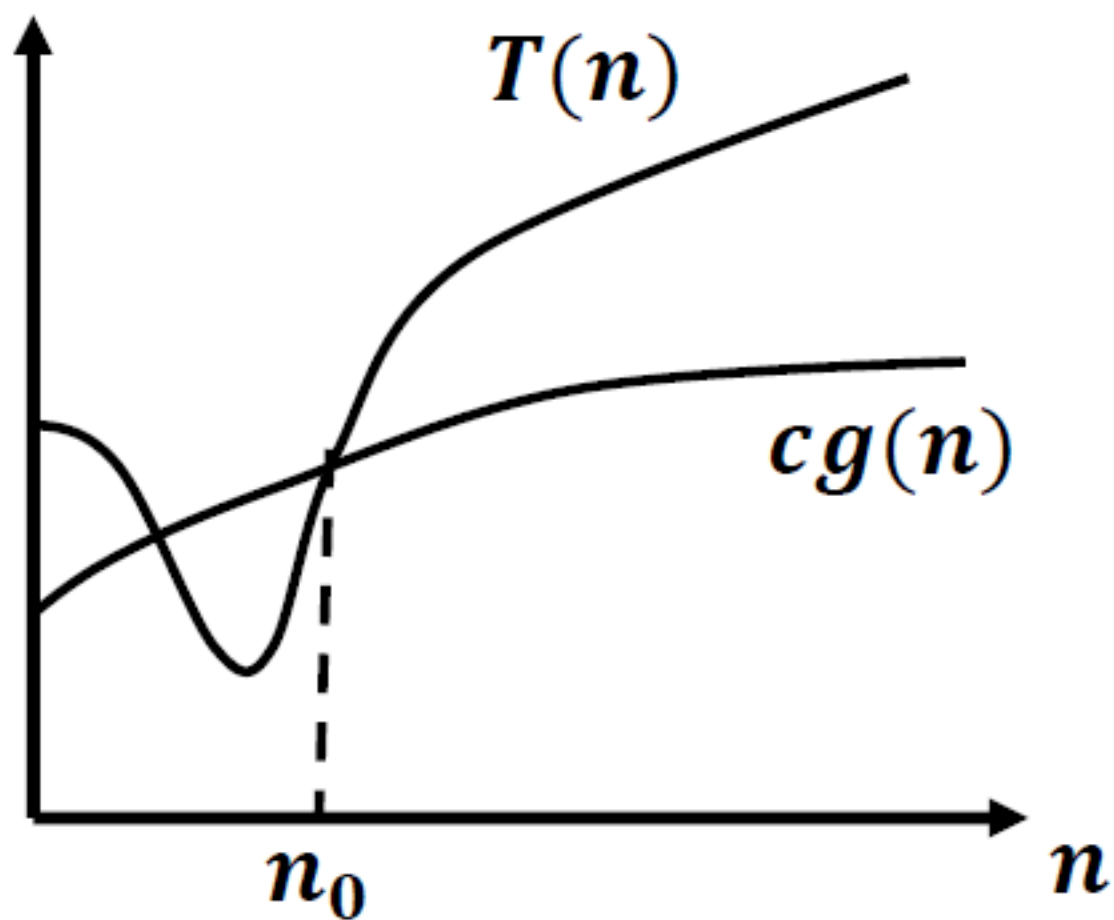
例子

设 $f(n) = n^2 + n$, 则

$f(n) = \Omega(n^2)$, 取 $c = 1, n_0 = 1$ 即可

$f(n) = \Omega(100n)$, 取 $c = 1/100, n_0 = 1$ 即可

1. $f(n) = \Omega(g(n))$, $f(n)$ 的阶不低于 $g(n)$ 的阶.
2. 可能存在多个正数 c , 指出一个即可.
3. 对前面有限个 n 值可以不满足上述不等式.



$$T(n) = \Omega(g(n))$$

1.3 数学基础：函数的渐近的界

定义1.1 设 f 和 g 是定义域为自然数集 N 上的函数.

- (1) 若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$ 成立, 则称 $f(n)$ 的渐近的上界是 $g(n)$, 记作 $f(n) = O(g(n))$.
- (2) 若存在正数 c 和 n_0 , 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$ 成立, 则称 $f(n)$ 的渐近的下界是 $g(n)$, 记作 $f(n) = \Omega(g(n))$.
- (3) 若对任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq f(n) < cg(n)$ 成立, 则记作 $f(n) = o(g(n))$.
- (4) 若对于任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq cg(n) < f(n)$ 成立, 则记作 $f(n) = \omega(g(n))$.
- (5) 若 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$, 则记作 $f(n) = \Theta(g(n))$.

例子

例子: $f(n)=n^2+n$, 则

$$f(n)=o(n^3)$$

$c \geq 1$ 显然成立, 因为 $n^2+n < cn^3$ ($n_0=2$)

任给 $1 > c > 0$, 取 $n_0 > \lceil 2/c \rceil$ 即可. 因为

$$cn \geq \underline{cn_0} > 2 \quad (\text{当 } n \geq n_0)$$

$$n^2+n < 2n^2 < cn^3$$

1. $f(n) = o(g(n))$, $f(n)$ 的阶低于 $g(n)$ 的阶
2. 对不同正数 c , n_0 不一样. c 越小 n_0 越大.
3. 对前面有限个 n 值可以不满足不等式.

1.3 数学基础：函数的渐近的界

定义1.1 设 f 和 g 是定义域为自然数集 N 上的函数.

- (1) 若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$ 成立, 则称 $f(n)$ 的渐近的上界是 $g(n)$, 记作 $f(n) = O(g(n))$.
- (2) 若存在正数 c 和 n_0 , 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$ 成立, 则称 $f(n)$ 的渐近的下界是 $g(n)$, 记作 $f(n) = \Omega(g(n))$.
- (3) 若对任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq f(n) < cg(n)$ 成立, 则记作 $f(n) = o(g(n))$.
- (4) 若对于任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq cg(n) < f(n)$ 成立, 则记作 $f(n) = \omega(g(n))$.
- (5) 若 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$, 则记作 $f(n) = \Theta(g(n))$.

例子

设 $f(n) = n^2 + n$, 则

$$f(n) = \omega(n),$$

不能写 $f(n) = \omega(n^2)$, 因为取 $c = 2$, 不存在 n_0 使得对一切 $n \geq n_0$ 有下式成立

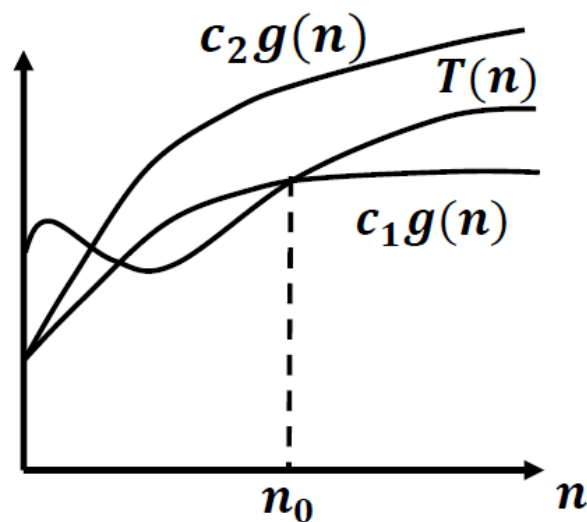
$$c n^2 = 2n^2 < n^2 + n \quad \times$$

1. $f(n) = \omega(g(n))$, $f(n)$ 的阶高于 $g(n)$ 的阶.
2. 对不同的正数 c , n_0 不等, c 越大 n_0 越大.
3. 对前面有限个 n 值可以不满足不等式.

1.3 数学基础：函数的渐近的界

定义1.1 设 f 和 g 是定义域为自然数集 N 上的函数.

- (1) 若存在正数 c 和 n_0 使得对一切 $n \geq n_0$ 有 $0 \leq f(n) \leq cg(n)$ 成立, 则称 $f(n)$ 的渐近的上界是 $g(n)$, 记作 $f(n) = O(g(n))$.
- (2) 若存在正数 c 和 n_0 , 使得对一切 $n \geq n_0$ 有 $0 \leq cg(n) \leq f(n)$ 成立, 则称 $f(n)$ 的渐近的下界是 $g(n)$, 记作 $f(n) = \Omega(g(n))$.
- (3) 若对任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq f(n) < cg(n)$ 成立, 则记作 $f(n) = o(g(n))$.
- (4) 若对于任意正数 c 都存在 n_0 , 使得当 $n \geq n_0$ 时有 $0 \leq cg(n) < f(n)$ 成立, 则记作 $f(n) = \omega(g(n))$.
- (5) 若 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$, 则记作 $f(n) = \Theta(g(n))$.



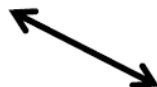
$$T(n) = \Theta(g(n))$$

例子: $f(n) = n^2 + n$, $g(n) = 100n^2$, 那么有

$$f(n) = \Theta(g(n))$$

1. $f(n)$ 的阶与 $g(n)$ 的阶相等.
2. 对前面有限个 n 值可以不满足条件.

渐近记号	名称
Θ	渐近紧确界
O	渐近上界
Ω	渐近下界



输入情况	情况说明
最好情况	不常出现，不具普遍性
最坏情况	确定上界，更具一般性

算法运行时间称为算法的时间复杂度，通常使用渐近记号 O 表示

例子：素数测试

算法 **PrimalityTest(n)**

输入： n , 大于2的奇整数

输出： true 或者 false

1. $s \leftarrow \lfloor n^{1/2} \rfloor$
2. for $j \leftarrow 2$ to s
3. if j 整除 n
4. then return false
5. return true

问题：

若 $n^{1/2}$ 可在 $O(1)$ 计算, 基本运算是整除, 以下表示是否正确?

$$W(n) = O(n^{1/2})$$



$$W(n) = \Theta(n^{1/2})$$



为什么?

有关定理

定理1.1 设 f 和 g 是定义域为自然数集合的函数.

(1) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在且等于某个常数 $c > 0$, 那么

$$f(n) = \Theta(g(n)).$$

(2) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, 那么 $f(n) = o(g(n))$.

(3) 如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$, 那么 $f(n) = \omega(g(n))$.

证明定理1.1 (1)

证 根据极限定义, 对于给定的正数 $\varepsilon = c/2$, 存在某个 n_0 , 只要 $n \geq n_0$, 就有

$$\begin{aligned} \left| \frac{f(n)}{g(n)} - c \right| < \varepsilon &\Rightarrow c - \varepsilon < \frac{f(n)}{g(n)} < c + \varepsilon \\ \Rightarrow \frac{c}{2} < \frac{f(n)}{g(n)} < \frac{3c}{2} < 2c \end{aligned}$$

对所有的 $n \geq n_0$, $f(n) \leq 2cg(n)$. 从而推出 $f(n) = O(g(n))$

对所有的 $n \geq n_0$, $f(n) \geq (c/2)g(n)$, 从而推出 $f(n) = \Omega(g(n))$,

于是 $f(n) = \Theta(g(n))$

例：估计函数的阶

例1 设 $f(n) = \frac{1}{2}n^2 - 3n$ ，证明 $f(n) = \Theta(n^2)$ 。

证 因为

$$\lim_{n \rightarrow \infty} \frac{f(n)}{n^2} = \lim_{n \rightarrow \infty} \frac{\frac{1}{2}n^2 - 3n}{n^2} = \frac{1}{2}$$

根据定理1，有 $f(n) = \Theta(n^2)$ 。

一些重要结果

可证明：多项式函数的阶低于指数函数的阶

$$n^d = o(r^n), \quad r > 1, \quad d > 0$$

证 不妨设 d 为正整数，

$$\begin{aligned}\lim_{n \rightarrow \infty} \frac{n^d}{r^n} &= \lim_{n \rightarrow \infty} \frac{dn^{d-1}}{r^n \ln r} = \lim_{n \rightarrow \infty} \frac{d(d-1)n^{d-2}}{r^n (\ln r)^2} \\ &= \dots = \lim_{n \rightarrow \infty} \frac{d!}{r^n (\ln r)^d} = 0\end{aligned}$$

分子分母
求导数

一些重要结果(续)

可证明：对数函数的阶低于幂函数的阶

$$\ln n = o(n^d), \quad d > 0$$

证

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\ln n}{n^d} &= \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{d n^{d-1}} \\ &= \lim_{n \rightarrow \infty} \frac{1}{d n^d} = 0 \end{aligned}$$

定理 2

定理 设函数 f, g, h 的定义域为自然数集合,

- (1) 如果 $f=O(g)$ 且 $g=O(h)$, 那么 $f=O(h)$.
- (2) 如果 $f=\Omega(g)$ 且 $g=\Omega(h)$, 那么 $f=\Omega(h)$.
- (3) 如果 $f=\Theta(g)$ 和 $g=\Theta(h)$, 那么 $f=\Theta(h)$

函数的阶之间的关系具有传递性

例子

按照阶从高到低排序以下函数：

$$f(n)=(n^2+n)/2, \quad g(n)=10n$$

$$h(n)=1.5^n, \quad t(n)=n^{1/2}$$

$$h(n) = \omega(f(n)),$$

$$f(n) = \omega(g(n)),$$

$$g(n) = \omega(t(n)),$$

排序 $h(n), f(n), g(n), t(n)$

基本函数类

阶的高低

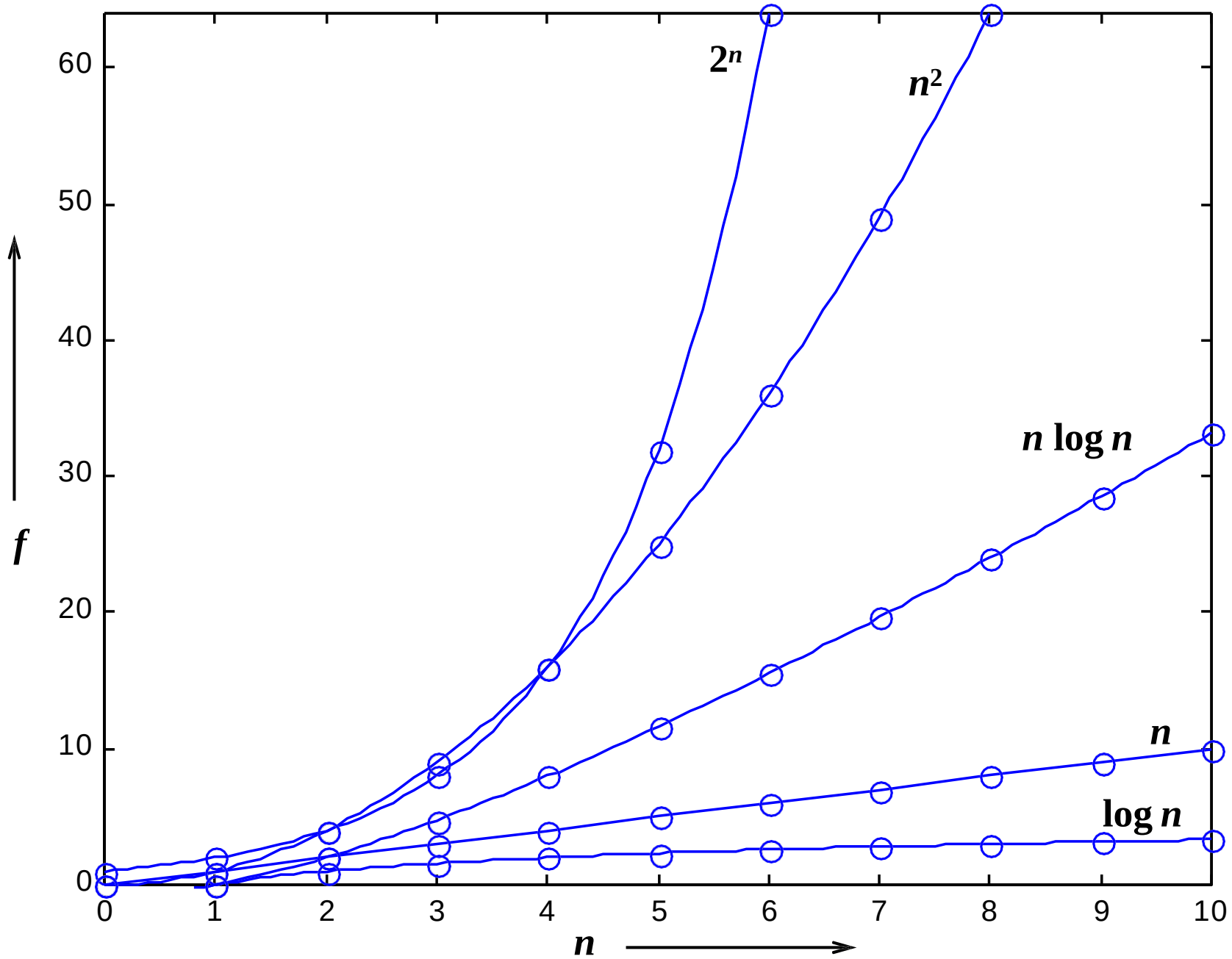
至少指数级: $2^n, 3^n, n!, \dots$

多项式级: $n, n^2, n \log n, n^{1/2}, \dots$

对数多项式级: $\log n, \log^2 n, \log \log n, \dots$

输入规模 n

函数	1	2	4	8	16	32
1	1	1	1	1	1	1
$\log n$	0	1	2	3	4	5
n	1	2	4	8	16	32
$n \log n$	0	2	8	24	64	160
n^2	1	4	16	64	256	1024
n^3	1	8	64	512	4096	32768
2^n	2	4	16	256	65536	4294967296
$n!$	1	2	24	40320	20922789888000	26313×10^{33}



	每秒 10 亿指令计算机的运行时间表						
n	$f(n)=n$	$n\log_2 n$	n^2	n^3	n^4	n^{10}	2^n
10	.01 μ s	.03 μ s	.1 μ s	1 μ s	10 μ s	10sec	1 μ s
20	.02 μ s	.09 μ s	.4 μ s	8 μ s	160 μ s	2.84hr	1ms
30	.03 μ s	.15 μ s	.9 μ s	27 μ s	810 μ s	6.83d	1sec
40	.04 μ s	.21 μ s	1.6 μ s	64 μ s	2.56ms	121.36d	18.3min
50	.05 μ s	.28 μ s	2.5 μ s	125 μ s	6.25ms	3.1yr	13d
100	.10 μ s	.66 μ s	10 μ s	1ms	100ms	3171yr	4*10 ¹³ yr
1,000	1.00 μ s	9.96 μ s	1ms	1sec	16.67min	3.17*10 ¹³ yr	32*10 ²⁸³ yr
10,000	10 μ s	130.03 μ s	100ms	16.67min	115.7d	3.17*10 ²³ yr	
100,000	100 μ s	1.66ms	10sec	11.57d	3171yr	3.17*10 ³³ yr	
1,000,000	1.0ms	19.92ms	16.67min	31.71yr	3.17*10 ⁷ yr	3.17*10 ⁴³ yr	

μ s = 微秒 = 10⁻⁶秒

ms = 毫秒 = 10⁻³秒

sec = 秒

min = 分钟

hr = 小时

d = 日

yr = 年

对数函数

符号:

$$\log n = \log_2 n$$

$$\log^k n = (\log n)^k$$

$$\log \log n = \log(\log n)$$

性质:

$$(1) \log_2 n = \Theta(\log_l n)$$

$$(2) \log_b n = o(n^\alpha) \quad \alpha > 0$$

$$(3) a^{\log_b n} = n^{\log_b a}$$

有关性质 (1) 的证明

$$\log_k n = \frac{\log_l n}{\log_l k} \quad \log_l k \text{ 为常数}$$

$$\lim_{n \rightarrow \infty} \frac{\log_k n}{\log_l n} = \lim_{n \rightarrow \infty} \frac{\log_l n}{\log_l k \cdot \log_l n} = \frac{1}{\log_l k}$$

根据定理, $\log_k n = \Theta(\log_l n)$

有关性质 (2) (3) 的说明

$$\log_b n = \Theta(\ln n)$$

$$\ln n = o(n^\alpha) \Rightarrow \log_b n = o(n^\alpha) \quad \alpha > 0$$

$$a^{\log_b n} = n^{\log_b a}$$



$$\log_b n \log_b a = \log_b a \log_b n$$

问题： m 元钱，投资 n 个项目. 效益函数 $f_i(x)$, 表示第 i 个项目投 x 元的效益, $i=1, 2, \dots, n$. 求如何分配每个项目的钱数使得总效益最大?

Stirling公式
$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + \Theta(\frac{1}{n}))$$

$$C_{m+n-1}^m = \frac{(m+n-1)!}{m!(n-1)!}$$

$$= \frac{\sqrt{2\pi(m+n-1)}(m+n-1)^{m+n-1}(1 + \Theta(\frac{1}{m+n-1}))}{\sqrt{2\pi m}m^m(1 + \Theta(\frac{1}{m}))\sqrt{2\pi(n-1)}(n-1)^{n-1}(1 + \Theta(\frac{1}{n-1}))}$$

$$= \Theta((1 + \varepsilon)^{m+n-1})$$

阶乘

Stirling公式
$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$$

$$n! = o(n^n)$$

$$n! = \Omega(2^n)$$

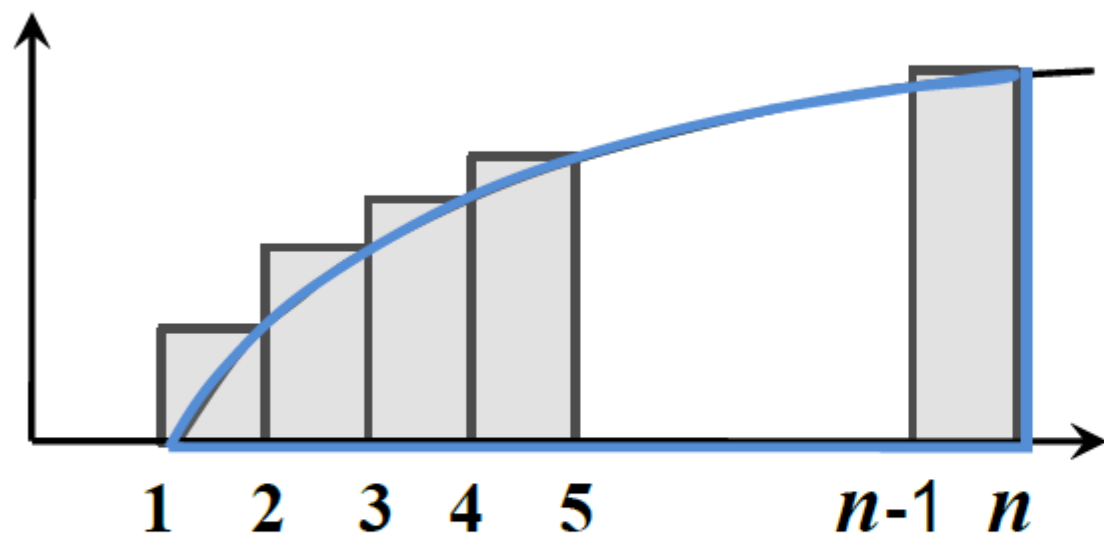
$$\log(n!) = \Theta(n \log n)$$

$$\lim_{n \rightarrow +\infty} \frac{\log(n!)}{n \log n} = \lim_{n \rightarrow +\infty} \frac{\ln(n!) / \ln 2}{n \ln n / \ln 2} = \lim_{n \rightarrow +\infty} \frac{\ln(n!)}{n \ln n}$$

$$= \lim_{n \rightarrow +\infty} \frac{\ln\left(\sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \left(\frac{c}{n}\right)\right)\right)}{n \ln n} = \lim_{n \rightarrow +\infty} \frac{\ln \sqrt{2\pi n} + n \ln \frac{n}{e}}{n \ln n} = 1$$

上述的 c 为某个常数

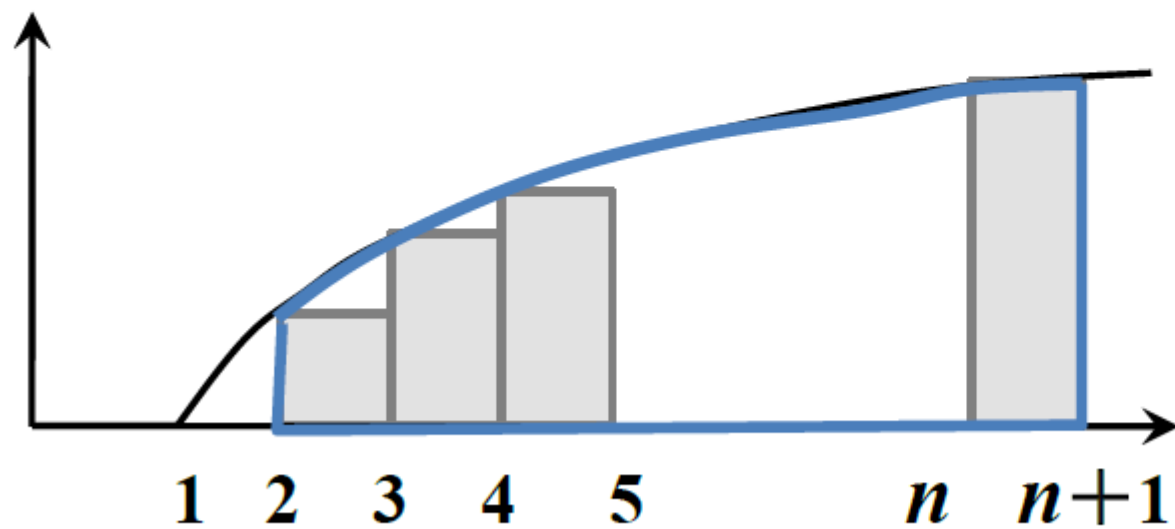
$\log(n!) = \Omega(n \log n)$ 的证明



$$\log(n!) = \sum_{k=1}^n \log k \geq \int_1^n \log x dx$$

$$= n \ln n - n + 1 = \Omega(n \log n)$$

$\log(n!) = O(n \log n)$ 的证明



$$\log(n!) = \sum_{k=1}^n \log k \leq \int_2^{n+1} \log x dx = O(n \log n)$$

如果 $f(k)$ 单调递增, 则 $\int_{m-1}^n f(x)dx \leq \sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x)dx$

当 $f(x)$ 单调递减时, $\int_m^{n+1} f(x)dx \leq \sum_{k=m}^n f(k) \leq \int_{m-1}^n f(x)dx$

函数的阶

例1.6 按照阶从高到低对以下函数排序：

$\log^2 n$, 1 , $n!$, $n2^n$, $n^{1/\log n}$, $(3/2)^n$, $\sqrt{\log n}$, $(\log n)^{\log n}$,
 2^{2^n} , $n^{\log \log n}$, n^3 , $\log \log n$, $n \log n$, n , $2^{\log n}$, $\log n$, $\log(n!)$

结果：

2^{2^n} , $n!$, $n2^n$, $(3/2)^n$, $(\log n)^{\log n} = n^{\log \log n}$,

n^3 , $\log(n!) = \Theta(n \log n)$, $n = \Theta(2^{\log n})$,

$\log^2 n$, $\log n$, $\sqrt{\log n}$, $\log \log n$,

$n^{1/\log n} = \Theta(1)$

注意



* 并非所有函数都是可比的，即对于函数 $f(n)$ 和 $g(n)$ ，可能 $f(n) \neq O(g(n))$, $f(n) \neq \Omega(g(n))$ 。例如， n 和 $n^{1+\sin n}$ 。

取整函数

取整函数的定义

$\lfloor x \rfloor$: 表示小于等于 x 的最大的整数

$\lceil x \rceil$: 表示大于等于 x 的最小的整数

实例

$$\lfloor 2.6 \rfloor = 2$$

$$\lceil 2.6 \rceil = 3$$

$$\lfloor 2 \rfloor = \lceil 2 \rceil = 2$$

应用: 二分检索

输入数组长度: n

中位数的位置: $\lfloor n/2 \rfloor$

与中位数比较后子问

题大小: $\lfloor n/2 \rfloor$

取整函数的性质

$$(1) \quad \underline{x-1} < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x+1$$

$$(2) \quad \underline{x+n} = \lfloor x \rfloor + n, \quad \lceil x+n \rceil = \lceil x \rceil + n, \quad n \text{ 为整数}$$

$$(3) \quad \left\lceil \frac{n}{2} \right\rceil + \left\lfloor \frac{n}{2} \right\rfloor = n$$

$$(4) \quad \left\lceil \frac{\left\lceil \frac{n}{a} \right\rceil}{b} \right\rceil = \left\lceil \frac{n}{ab} \right\rceil, \quad \left\lfloor \frac{\left\lfloor \frac{n}{a} \right\rfloor}{b} \right\rfloor = \left\lfloor \frac{n}{ab} \right\rfloor$$

求和

- 几个有用的结果

$$\sum_{k=1}^n a_k = \frac{n(a_1 + a_n)}{2}$$

$$\sum_{k=0}^n a q^k = \frac{a(1 - q^{n+1})}{1 - q}, \quad \sum_{k=0}^n x^k = \frac{1 - x^{n+1}}{1 - x}$$

$$\sum_{k=1}^n \frac{1}{k} = \ln n + O(1)$$

- 和的上界 $\sum_{k=1}^n a_k \leq n a_{\max}$

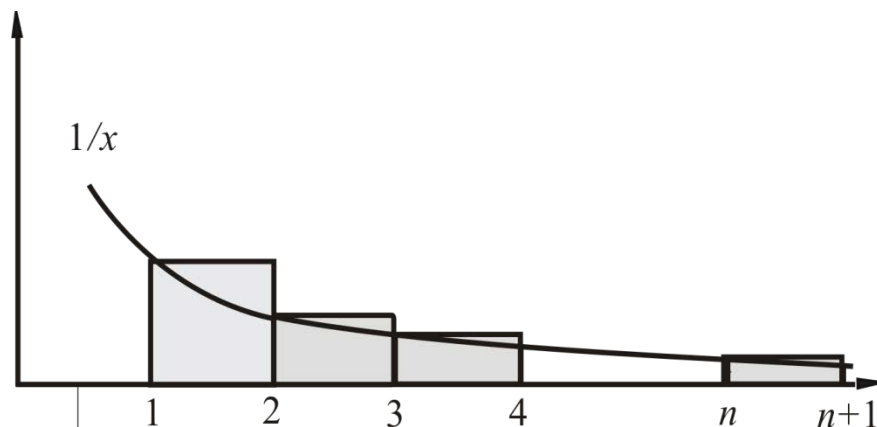
- 假设存在常数 $r < 1$, 使得 对一切 $k \geq 0$ $\frac{a_{k+1}}{a_k} \leq r$ 成立, 则

$$\sum_{k=0}^n a_k \leq \sum_{k=0}^{\infty} a_0 r^k = a_0 \sum_{k=0}^{\infty} r^k = \frac{a_0}{1 - r}$$

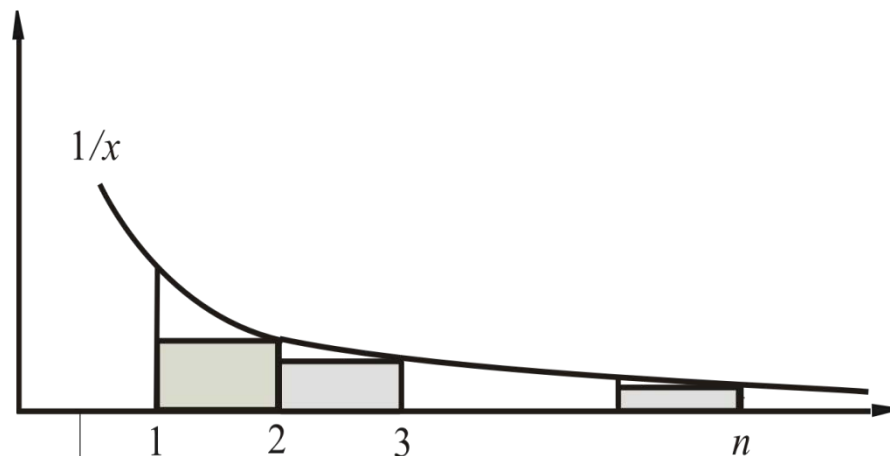
估计和式渐近的界

估计 $\sum_{k=1}^n \frac{1}{k}$ 的渐近的界.

$$\begin{aligned}\sum_{k=1}^n \frac{1}{k} &\geq \int_1^{n+1} \frac{dx}{x} \\ &= \ln(n+1)\end{aligned}$$



$$\begin{aligned}\sum_{k=1}^n \frac{1}{k} &= 1 + \sum_{k=2}^n \frac{1}{k} \\ &\leq 1 + \int_1^n \frac{dx}{x} \\ &= \ln n + 1\end{aligned}$$



求和实例

例1.7 求和

$$(1) \sum_{k=1}^{n-1} \frac{1}{k(k+1)} \quad (2) \sum_{t=1}^k t 2^{t-1}$$

解

$$\begin{aligned} (1) \quad \sum_{k=1}^{n-1} \frac{1}{k(k+1)} &= \sum_{k=1}^{n-1} \left(\frac{1}{k} - \frac{1}{k+1} \right) \\ &= \sum_{k=1}^{n-1} \frac{1}{k} - \sum_{k=1}^{n-1} \frac{1}{k+1} = \sum_{k=1}^{n-1} \frac{1}{k} - \sum_{k=2}^n \frac{1}{k} = 1 - \frac{1}{n} \end{aligned}$$

$$\begin{aligned} (2) \quad \sum_{t=1}^k t 2^{t-1} &= \sum_{t=1}^k t(2^t - 2^{t-1}) = \sum_{t=1}^k t 2^t - \sum_{t=1}^k t 2^{t-1} = \sum_{t=1}^k t 2^t - \sum_{t=0}^{k-1} (t+1) 2^t \\ &= \sum_{t=1}^k t 2^t - \sum_{t=0}^{k-1} t 2^t - \sum_{t=0}^{k-1} 2^t = k 2^k - (2^k - 1) = (k-1) 2^k + 1 \end{aligned}$$

求和实例

例1.8 估计 $\sum_{k=1}^n \frac{k}{3^k}$ 的上界.

解 由

$$a_k = \frac{k}{3^k}, \quad a_{k+1} = \frac{k+1}{3^{k+1}}$$

得

$$\frac{a_{k+1}}{a_k} = \frac{1}{3} \frac{k+1}{k} \leq \frac{2}{3}$$

$$\sum_{k=1}^n \frac{k}{3^k} \leq \sum_{k=1}^{\infty} \frac{1}{3} \left(\frac{2}{3}\right)^{k-1} = \frac{1}{3} \frac{1}{1 - \frac{2}{3}} = 1$$

二分检索算法

算法 BinarySearch (T, l, r, x)

输入：数组 T ，下标从 l 到 r ；数 x

输出： j

1. $l \leftarrow 1; r \leftarrow n$

2. while $l \leq r$ do

3. $m \leftarrow \lfloor (l+r)/2 \rfloor$

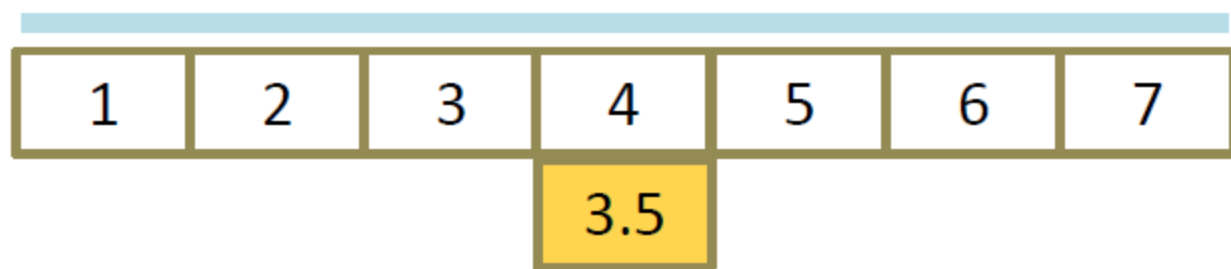
4. if $T[m]=x$ then return m // x 中位元素

5. else if $T[m] > x$ then $r \leftarrow m-1$

6. else $l \leftarrow m+1$

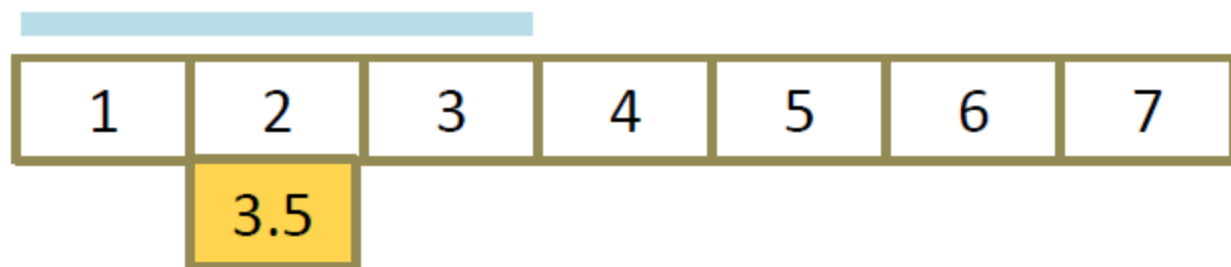
7. return 0

二分检索运行实例



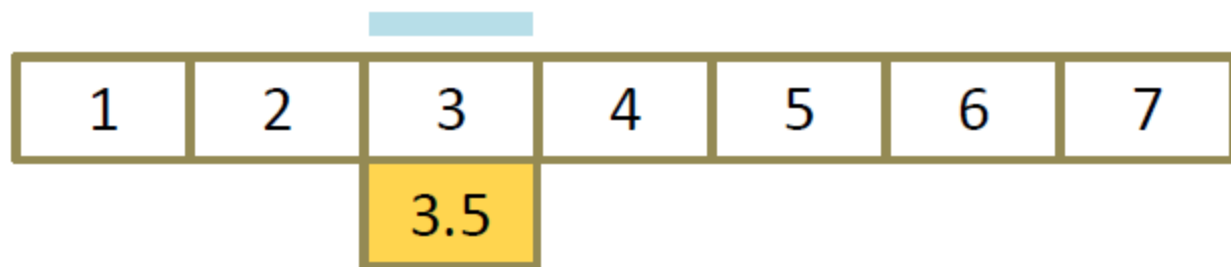
1	2	3	4	5	6	7
			3.5			

第1次
 $3.5 < 4$



1	2	3	4	5	6	7
	3.5					

第2次
 $3.5 > 2$



1	2	3	4	5	6	7
		3.5				

第3次
 $3.5 > 3$

$2n+1$ 个输入

假设 $n = 2^k - 1$, 输入有 $2n + 1$ 种:

$$x = T[1]$$

$$x = T[2]$$

...

$$x = T[n-1]$$

$$x = T[n]$$

x 在 T 中

$$x < T[1]$$

$$T[1] < x < T[2]$$

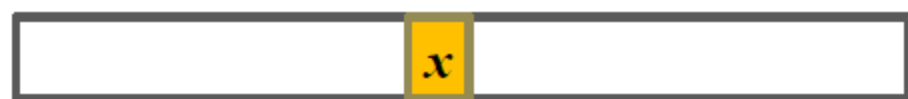
...

$$T[n-1] < x < T[n]$$

$$T[n] < x$$

x 不在 T 中

比较 t 次的输入个数



比较1次:1个



比较2次:2个



比较3次:4个

对 $t=1, 2, \dots, k-1$, 比较 t 次: 2^{t-1} 个

比较 k 次的输入有 $2^{k-1} + n + 1$ 个

总次数: 对每个输入乘以次数并求和

二分检索平均时间复杂度

假设 $n = 2^k - 1$ ，各种输入概率相等

$$\begin{aligned} A(n) &= \frac{1}{2n+1} \left[\sum_{t=1}^{k-1} t 2^{t-1} + k(2^{k-1} + n + 1) \right] \\ &= \frac{1}{2n+1} \left[\sum_{t=1}^k t 2^{t-1} + k(n+1) \right] \\ &= \frac{1}{2n+1} \left[(k-1)2^k + 1 + k(n+1) \right] \\ &= \frac{k2^k - 2^k + 1 + k2^k}{2^{k+1} - 1} \approx k - \frac{1}{2} = \lfloor \log n \rfloor + \frac{1}{2} \end{aligned}$$

递推方程

设序列 $a_0, a_1, \dots, a_n, \dots$, 简记为 $\{a_n\}$, 一个把 a_n 与某些个 $a_i (i < n)$ 联系起来的等式叫做关于序列 $\{a_n\}$ 的递推方程

递推方程的例子

Fibonacci数

1, 1, 2, 3, 5, 8, 13, 21,
34, 55, ...

递推方程:

$$f_n = f_{n-1} + f_{n-2}$$

初值:

$$f_0 = 1, f_1 = 1$$

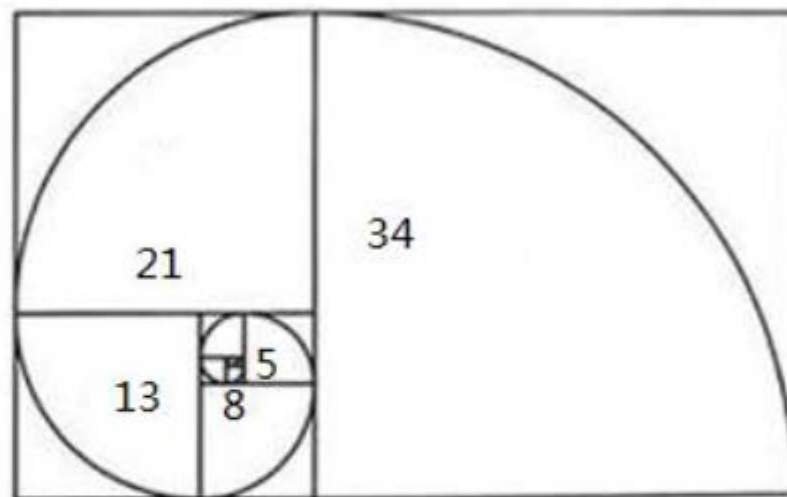
解:

$$f_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1}$$



数学家Fibonacci
意大利1170-1240

Fibonacci数的存在



55



递推方程的求解

求解方法：

- 迭代法

直接迭代：插入排序最坏情况下时间分析

换元迭代：二分归并排序最坏情况下时间分析

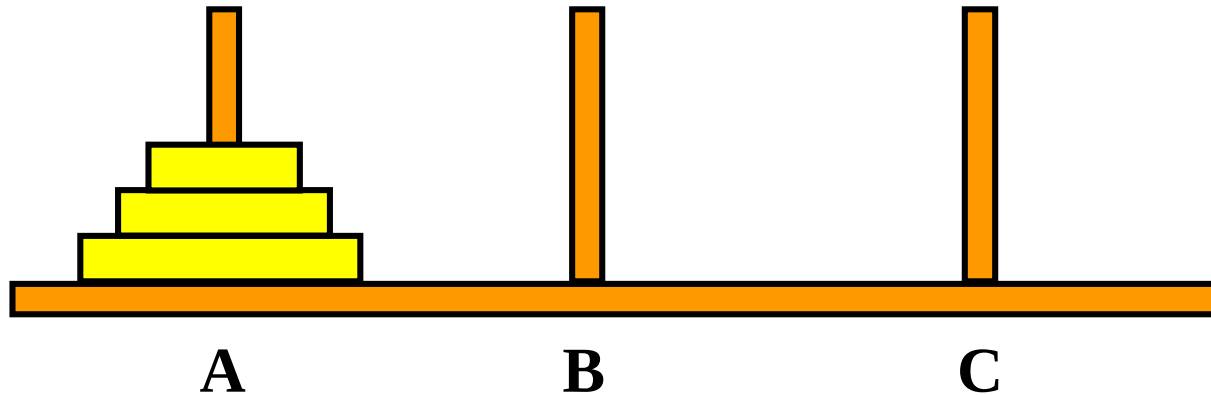
差消迭代：快速排序平均情况下的时间分析

迭代模型：递归树

- 尝试法：快速排序平均情况下的时间分析

- 主定理：递归算法的分析

例1.10: Hanoi塔



算法1.3 $\text{Hanoi}(A, C, n)$ // 将A的 n 个盘子按要求移到C

1. if $n=1$ then move (A, C) // 将A的1个盘子移到C

2. else $\text{Hanoi}(A, B, n-1)$

3. move (A, C)

4. $\text{Hanoi}(B, C, n-1)$

$T(n) = 2 T(n-1) + 1$, $T(1) = 1$, 迭代解得 $T(n) = 2^n - 1$

1秒移1个, 64个盘子要多少时间? (5000亿年)

迭代法

- 不断用递推方程的右部替换左部
- 每次替换，随着 n 的降低在和式中多出一项
- 直到出现初值停止迭代
- 将初值代入并对和式求和
- 可用数学归纳法验证解的正确性

直接迭代：插入排序

算法1.4 InsertSort(A, n) // A 为 n 个数的数组

```
1. for  $j \leftarrow 2$  to  $n$  do
2.    $x \leftarrow A[j]$ 
3.    $i \leftarrow j-1$            // 行3到行7把 $A[j]$ 插入 $A[1..j-1]$ 之中
4.   while  $i > 0$  and  $x < A[i]$  do
5.      $A[i+1] \leftarrow A[i]$ 
6.      $i \leftarrow i-1$ 
7.    $A[i+1] \leftarrow x$ 
```

$$\begin{cases} W(n) = W(n-1) + n - 1 \\ W(1) = 0 \end{cases}$$

$$W(n) = W(n-1) + n - 1$$

$$= [W(n-2) + n - 2] + n - 1 = W(n-2) + (n-2) + (n-1)$$

$$= [W(n-3) + n - 3] + (n-2) + (n-1) = \dots$$

$$= W(1) + 1 + 2 + \dots + (n-2) + (n-1)$$

$$= 1 + 2 + \dots + (n-2) + (n-1) = n(n-1)/2$$

解的正确性-归纳验证

证明:下述递推方程的解是 $W(n)=n(n-1)/2$

$$W(n)=W(n-1)+n-1$$

$$W(1)=0$$

方法: 数学归纳法

证 $n=1$, $W(1)=1 \times (1-1)/2 = 0$

假设对于 n , 解满足方程, 则

$$\begin{aligned} & W(n+1) \\ &= \underline{W(n)} + n = \underline{n(n-1)/2} + n \\ &= n[(n-1)/2 + 1] = n(n+1)/2 \end{aligned}$$

换元迭代

- 将对 n 的递推式换成对其他变元 k 的递推式
- 对 k 直接迭代
- 将解 (关于 k 的函数) 转换成关于 n 的函数

二分归并排序

算法1.5 MergeSort(A, p, r) // 归并排序数组 $A[p..r]$, $1 \leq p \leq r \leq n$

1. if $p < r$
2. then $q \leftarrow \lfloor (p+r)/2 \rfloor$
3. MergeSort(A, p, q)
4. MergeSort($A, q+1, r$)
5. Merge(A, p, q, r)

归并过程

Merge(A,p,q,r) // 将排序数组A[p..q]与A[q+1,r]合并

1. $x \leftarrow q - p + 1, y \leftarrow r - q$ // x, y分别为两个子数组的元素数
2. 将A[p..q]复制到B[1..x], 将A[q+1..r]复制到C[1..y]
3. $i \leftarrow 1, j \leftarrow 1, k \leftarrow p$
4. While $i \leq x$ and $j \leq y$ do
 5. if $B[i] \leq C[j]$ // B的首元素不大于C的首元素
 6. then $A[k] \leftarrow B[i]$ // 将B的首元素放到A中
 7. $i \leftarrow i + 1$
 8. else
 9. $A[k] \leftarrow C[j]$
 10. $j \leftarrow j + 1$
 11. $k \leftarrow k + 1$
12. if $i > x$ then 将C[j..y]复制到A[k..r] // B已经是空数组
13. else 将B[i..x]复制到A[k..r] // C已经是空数组

$$\begin{cases} W(n) = 2W(n/2) + n - 1 \\ W(1) = 0 \end{cases}$$

换元

假设 $n=2^k$, 递推方程如下:

$$W(n)=2W(n/2)+n-1$$

$$W(1)=0$$

换元:

$$W(2^k) = 2W(2^{k-1}) + 2^k - 1$$

$$W(0) = 0$$

迭代求解

$$W(2^k) = 2W(2^{k-1}) + 2^k - 1$$

$$W(n) = 2W(2^{k-1}) + 2^k - 1$$

$$= 2[2W(2^{k-2}) + 2^{k-1} - 1] + 2^k - 1$$

$$= 2^2 W(2^{k-2}) + 2^k - 2 + 2^k - 1$$

$$= 2^2 [2W(2^{k-3}) + 2^{k-2} - 1] + 2^k - 2 + 2^k - 1$$

$$= \dots$$

$$= 2^k W(1) + k2^k - (2^{k-1} + 2^{k-2} + \dots + 2 + 1)$$

$$= k2^k - 2^k + 1$$

$$= n \log n - n + 1$$

换元，
对k迭代

使用迭代法，对解可以通过数学归纳法验证

快速排序

- 假设 $A[p..r]$ 的元素彼此不等
以首元素 $A[p]$ 对数组 $A[p..r]$ 划分,使得:
 小于 x 的元素放在 $A[p..q-1]$
 大于 x 的元素放在 $A[q+1..r]$
- 递归对 $A[p..q-1]$ 和 $A[q+1..r]$ 排序

工作量: 子问题工作量+划分工作量

输入情况

- 有 n 种可能的输入

x 排好序位置	子问题 1 规模	子问题 2 规模
1	0	$n-1$
2	1	$n-2$
3	2	$n-3$
...	...	...
$n-1$	$n-2$	1
n	$n-1$	0

对每个输入，划分的比较次数都是 $n-1$

工作量总和

$$T(0) + T(n-1) + n-1$$

$$T(1) + T(n-2) + n-1$$

$$T(2) + T(n-3) + n-1$$

...

$$+ T(n-1) + T(0) + n-1$$

$$2[T(1)+...+T(n-1)]+n(n-1)$$

快速排序平均工作量

假设首元素排好序在每个位置是等概率的

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + O(n), n \geq 2$$

$$T(1) = 0$$

全部历史递推方程

对于高阶方程应该先化简，然后迭代

差消化简

利用两个方程相减，将右边的项尽可能消去，以达到降阶的目的

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + cn$$

$$nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2$$

$$(n-1)T(n-1) = 2 \sum_{i=1}^{n-2} T(i) + c(n-1)^2$$

差消化简

$$\begin{aligned} & nT(n) - \underline{(n-1)T(n-1)} \\ = & \underline{2T(n-1)} + cn^2 - c(n-1)^2 \end{aligned}$$



$$nT(n) = (n+1)T(n-1) + c_1n$$



$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{c_1}{n+1}$$

迭代求解

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \boxed{\frac{c_1}{n+1}} = \dots$$

$$= c_1 \left[\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} \right] + \frac{T(1)}{2}$$

$$= c_1 \left[\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} \right]$$

代入
初值

$$= \Theta(\log n)$$

$$T(n) = \Theta(n \log n)$$

递归树的概念

- 递归树是迭代计算的模型.
- 递归树的生成过程与迭代过程一致.
- 递归树上所有项恰好是迭代之后产生和式中的项.
- 对递归树上的项求和就是迭代后方程的解.

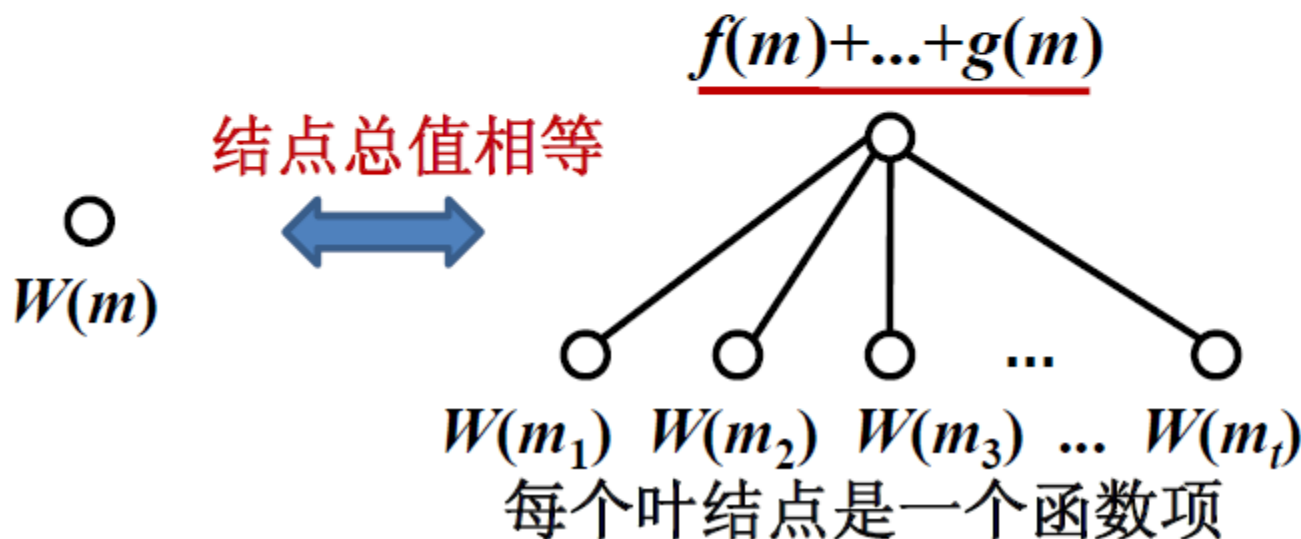
迭代在递归树中的表示

如果递归树上某结点标记为 $W(m)$

$$W(m) = W(m_1) + \dots + W(m_t)$$

$$+ \underline{f(m) + \dots + g(m)}, \quad m_1, \dots, m_t < m$$

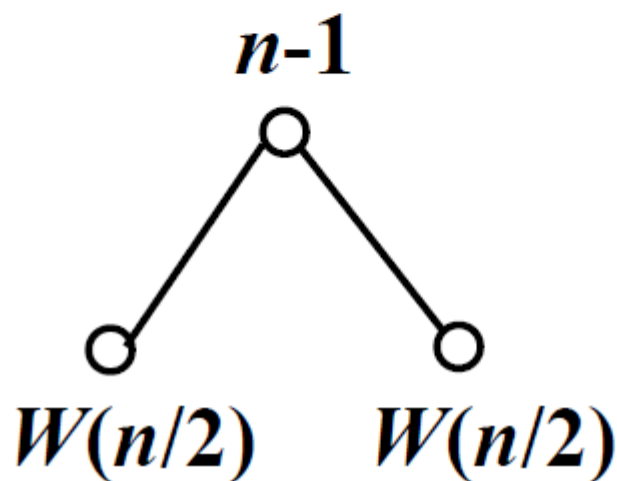
其中 $W(m_1), \dots, W(m_t)$ 称为函数项。



二层子树的例子

二分归并排序

$$W(n) = 2W(n/2) + \underline{n-1}$$



递归树的生成规则

- 初始，递归树只有根结点，其值为 $W(n)$
- 不断继续下述过程：
 - 将函数项叶结点的迭代式 $W(m)$ 表示成二层子树
 - 用该子树替换该叶结点
- 继续递归树的生成，直到树中无函数项(只有初值)为止.

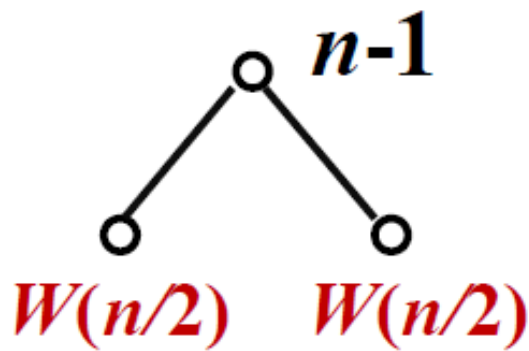
递归树生成实例

$$W(n) = 2W(n/2) + n-1$$

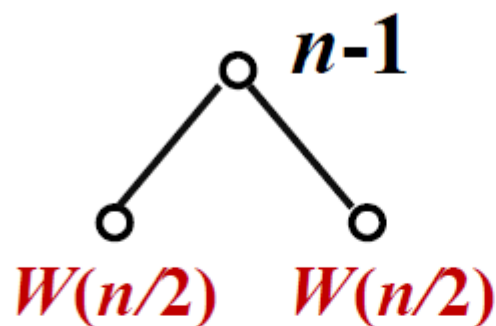
$W(n)$ 初始
○



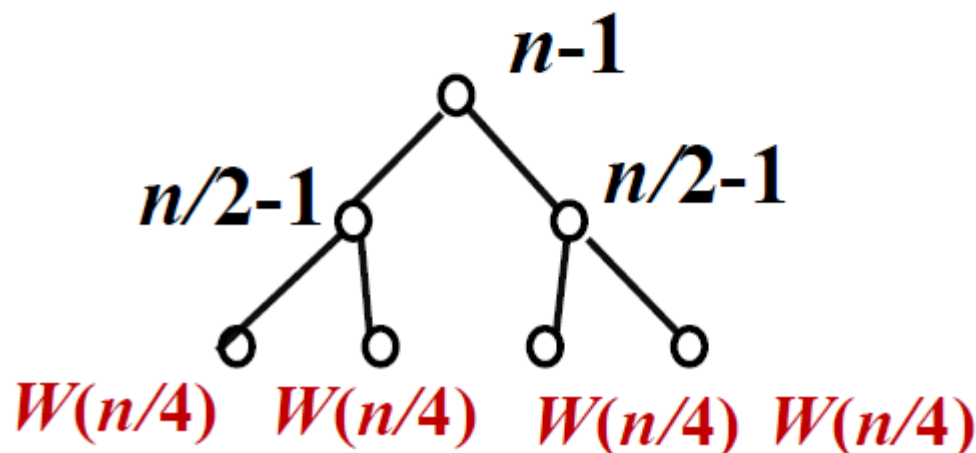
第一步迭代



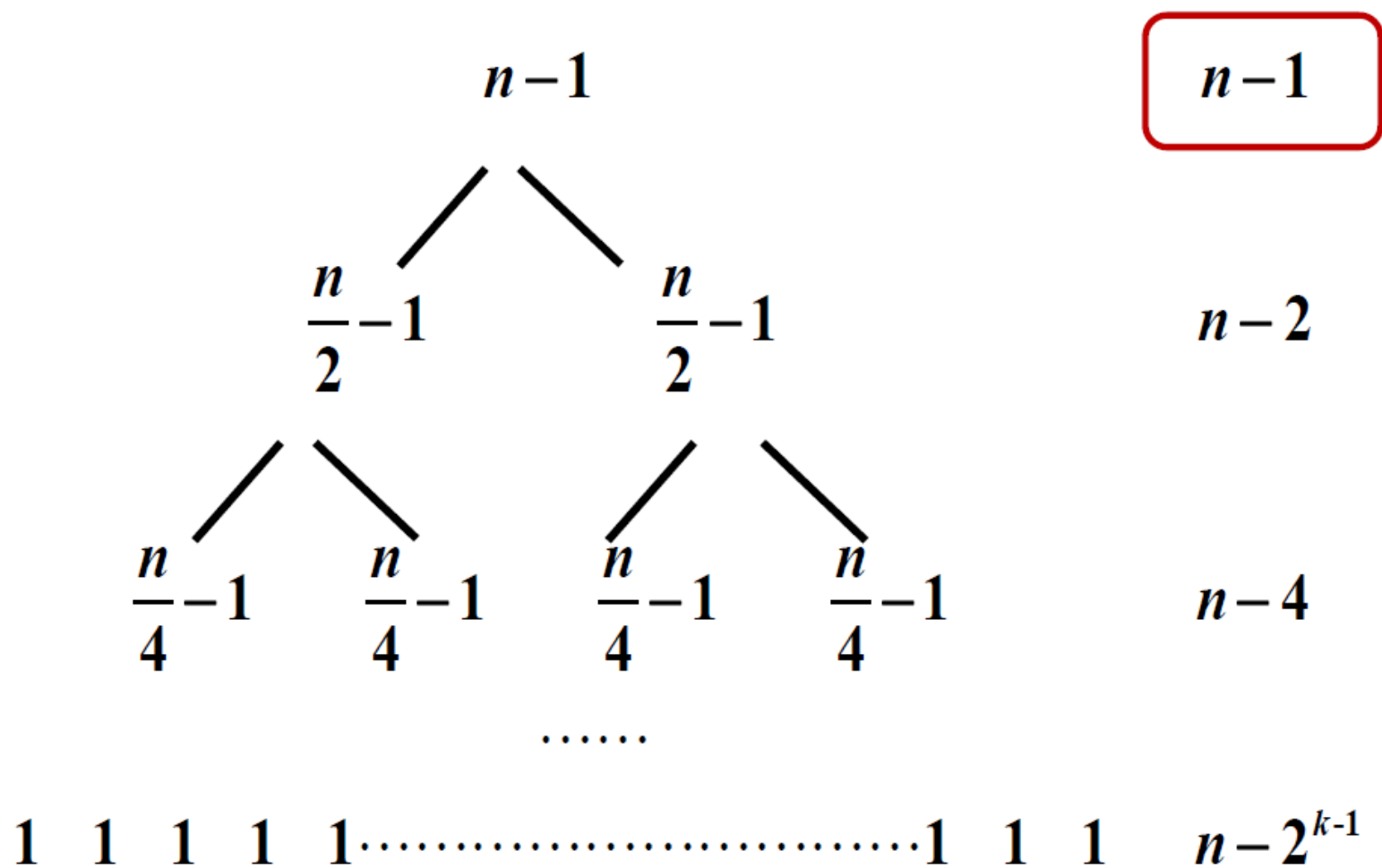
递归树生成实例



第二步迭代



递归树



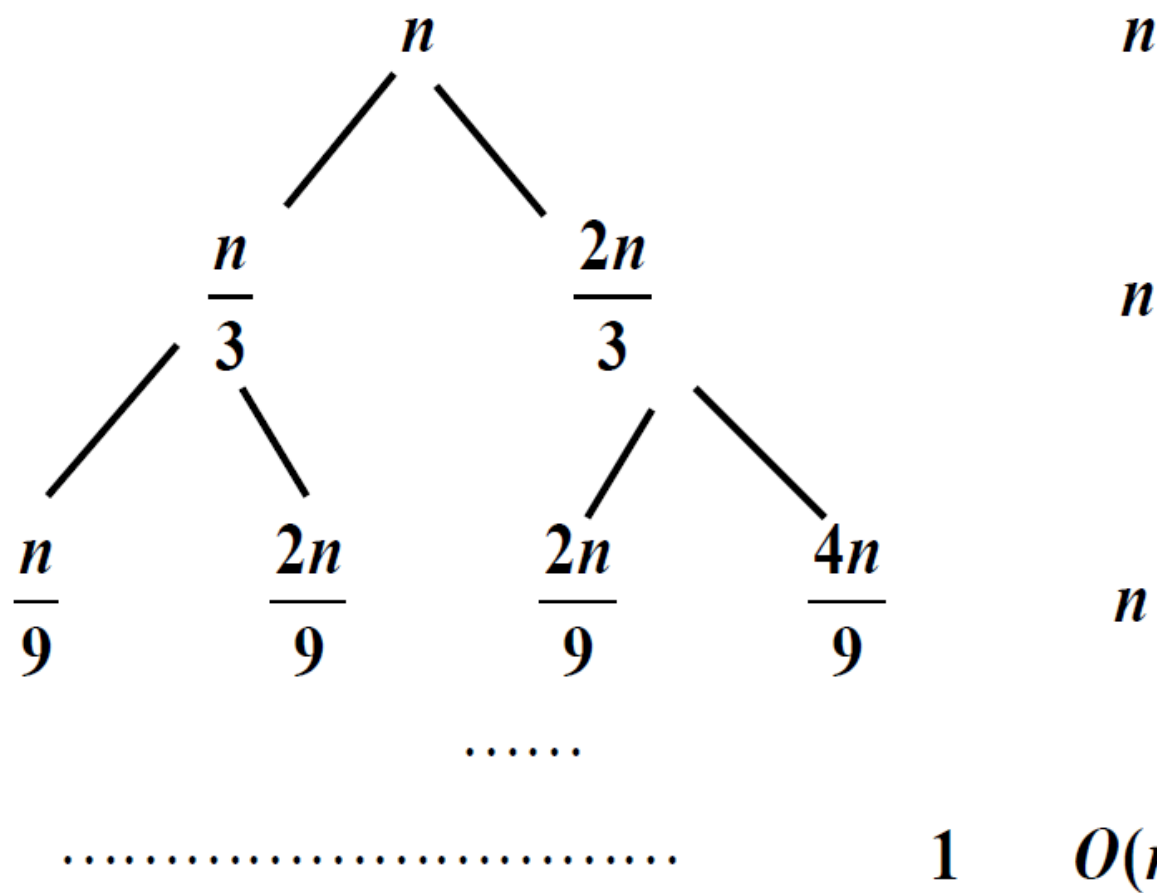
对递归树上的量求和

$$\begin{aligned} W(n) &= 2W(n/2) + n - 1, \quad n = 2^k, \\ W(1) &= 0 \end{aligned}$$

$$\begin{aligned} W(n) &= n - 1 + n - 2 + \dots + n - 2^{k-1} \\ &= kn - (2^k - 1) \\ &= n \log n - n + 1 \end{aligned}$$

递归树应用实例

求解方程: $T(n) = T(n/3) + T(2n/3) + n$



求和

方程: $T(n)=T(n/3)+T(2n/3)+n$

递归树层数 k , 每层 $O(n)$

$$n(2/3)^k = 1$$

$$\Rightarrow (3/2)^k = n$$

$$\Rightarrow k = O(\log_{3/2} n)$$

$$T(n)=O(n\log n)$$

尝试法：快速排序

(1) $T(n)=C$ 为常函数, 左边= $O(1)$

$$\text{右边} = \frac{2}{n}C(n-1) + O(n) = 2C - \frac{2C}{n} + O(n)$$

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + O(n)$$

(2) $T(n)=cn$, 左边= cn

$$\text{右边} = \frac{2}{n} \sum_{i=1}^{n-1} ci + O(n) = \frac{2c}{n} \frac{(1+n-1)(n-1)}{2} + O(n) = cn - c + O(n)$$

(3) $T(n)=cn^2$, 左边= cn^2

$$\text{右边} = \frac{2}{n} \sum_{i=1}^{n-1} ci^2 + O(n) = \frac{2}{n} \left[\frac{cn^3}{3} + O(n^2) \right] + O(n) = \frac{2c}{3}n^2 + O(n)$$

(4) $T(n)=cn \log n$, 左边= $cn \log n$

$$\text{右边} = \frac{2c}{n} \sum_{i=1}^{n-1} i \log i + O(n) = cn \log n + O(n)$$

解: $T(n) = \Theta(n \log n)$

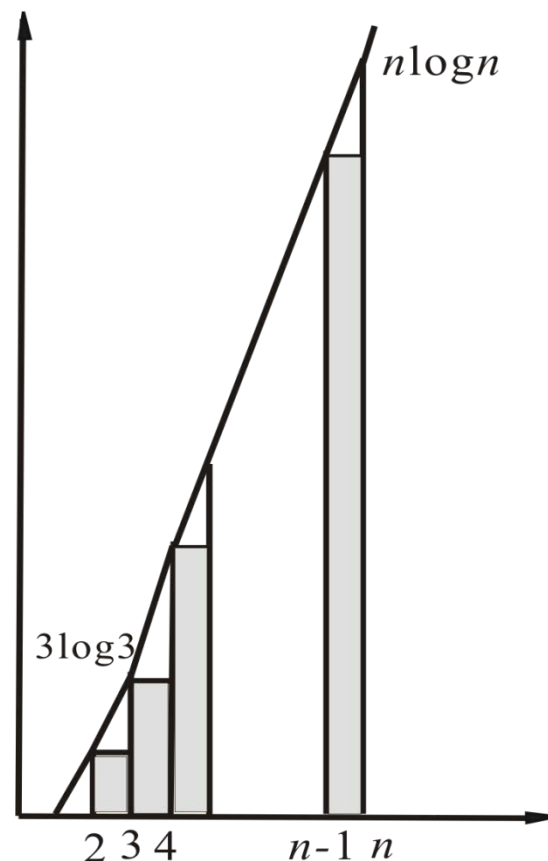
以积分作为求和的近似

$$\int_1^{n-1} x \log x dx \leq \sum_{i=1}^{n-1} i \log i \leq \int_2^n x \log x dx$$

$$\int_2^n x \log x dx = \int_2^n \frac{x}{\ln 2} \ln x dx$$

$$= \frac{1}{\ln 2} \left[\frac{x^2}{2} \ln x - \frac{x^2}{4} \right] \Big|_2^n$$

$$= \frac{1}{\ln 2} \left(\frac{n^2}{2} \ln n - \frac{n^2}{4} \right) - \frac{1}{\ln 2} \left(\frac{4}{2} \ln 2 - \frac{4}{4} \right)$$



主定理的应用背景

求解递推方程

$$T(n) = a T(n/b) + f(n)$$

a : 归约后的子问题个数

n/b : 归约后子问题的规模

$f(n)$: 归约过程及组合子问题的解的工作量

二分检索: $T(n) = T(n/2) + 1$

二分归并排序: $T(n) = 2T(n/2) + n - 1$

主定理

定理1.6 设 $a \geq 1, b > 1$ 为常数, $f(n)$ 为函数, $T(n)$ 为非负整数, 且

$$T(n) = aT(n/b) + f(n)$$

则有以下结果:

1. 若 $f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$
2. 若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \log n)$
3. 若 $f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0$, 且对某个常数 $c < 1$ 和所有充分大的 n 有 $a f(n/b) \leq c f(n)$, 那么

$$T(n) = \Theta(f(n))$$

迭代

$$T(n) = aT(n/b) + f(n)$$

设 $n = b^k$

$$T(n) = aT(\frac{n}{b}) + f(n)$$

$$= a[aT(\frac{n}{b^2}) + f(\frac{n}{b})] + f(n)$$

$$= a^2T(\frac{n}{b^2}) + af(\frac{n}{b}) + f(n)$$

$$= \dots$$

迭代结果

$$a^{\log_b n} = n^{\log_b a}$$

$$= a^k T\left(\frac{n}{b^k}\right) + a^{k-1} f\left(\frac{n}{b^{k-1}}\right) + \dots + a f\left(\frac{n}{b}\right) + f(n)$$

$$= \underline{a^k T(1)} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$= \underline{c_1 n^{\log_b a}} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \quad T(1) = c_1$$

- 第一项为所有最小子问题的计算工作量
- 第二项为迭代过程归约到子问题及综合解的工作量

情况1

$$f(n) = O(n^{\log_b a - \varepsilon})$$

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{\log_b a - \varepsilon})^j}\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1}\right) \\ &= c_1 n^{\log_b a} + O(n^{\log_b a - \varepsilon} n^\varepsilon) = \Theta(n^{\log_b a}) \end{aligned}$$

情况2 $f(n) = \Theta(n^{\log_b a})$

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + \Theta\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\ &= c_1 n^{\log_b a} + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{a^j}\right) \\ &= c_1 n^{\log_b a} + \Theta(n^{\log_b a} \log n) \\ &= \Theta(n^{\log_b a} \log n) \end{aligned}$$

情况3

$$f(n) = \Omega(n^{\log_b a + \varepsilon}) \quad (1)$$

$$af(n/b) \leq cf(n) \quad (2)$$

$$T(n) = c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$\leq c_1 n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} c^j f(n) \quad \left(af\left(\frac{n}{b}\right) \leq cf(n)\right)$$

$$= c_1 n^{\log_b a} + f(n) \frac{c^{\log_b n} - 1}{c - 1}$$

$$= c_1 n^{\log_b a} + \Theta(f(n)) \quad (c < 1)$$

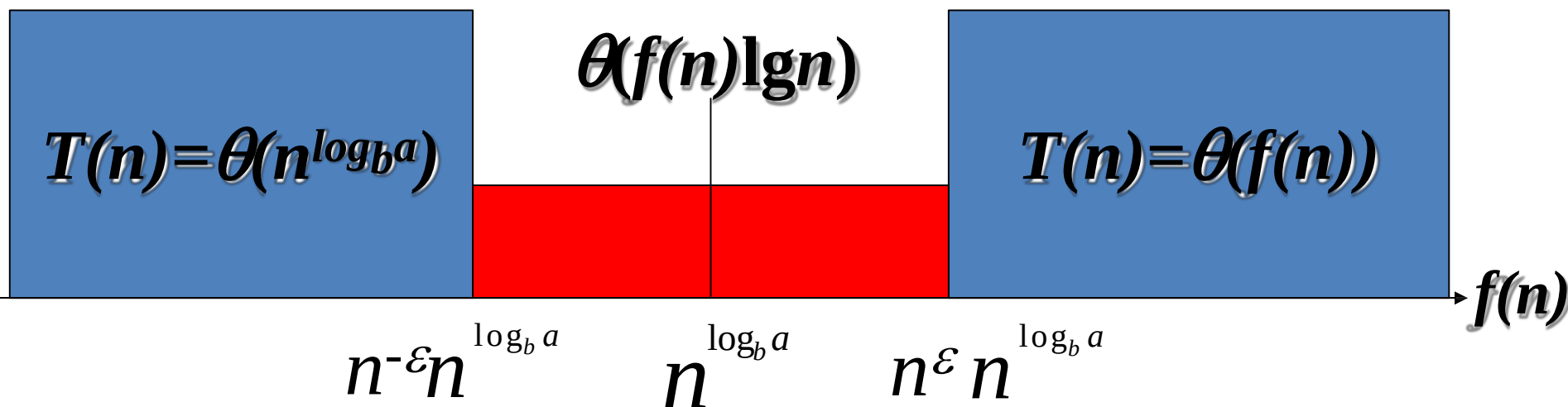
$$= \Theta(f(n)) \quad (f(n) = \Omega(n^{\log_b a + \varepsilon}))$$

*直观地：我们用 $f(n)$ 与 $n^{\log_b a}$ 比较

(1). 若 $n^{\log_b a}$ 大, 则 $T(n) = \theta(n^{\log_b a})$

(2). 若 $f(n)$ 大, 则 $T(n) = \theta(f(n))$

(3). 若 $f(n)$ 与 $n^{\log_b a}$ 同阶, 则 $T(n) = \theta(n^{\log_b a} \lg n) = \theta(f(n) \lg n)$.



对于红色部分，主定理无能为力

更进一步:

- (1). 在第一种情况, $f(n)$ 不仅小于 $n^{\log_b a}$, 必须多项式地小于, 即对于一个常数 $\varepsilon > 0$, $f(n) = O\left(\frac{n^{\log_b a}}{n^\varepsilon}\right)$.
- (2). 在第三种情况, $f(n)$ 不仅大于 $n^{\log_b a}$, 必须多项式地大于, 即对一个常数 $\varepsilon > 0$, $f(n) = \Omega(n^{\log_b a} \cdot n^\varepsilon)$.

主定理的应用

例1.16 求解递推方程 $T(n) = 9T(n/3) + n$

解 上述递推方程中的 $a = 9, b = 3, f(n) = n$, 那么

$$n^{\log_3 9} = n^2, \quad f(n) = O(n^{\log_3 9 - 1}),$$

相当于主定理的第一种情况, 其中 $\varepsilon = 1$. 根据定理得到

$$T(n) = \Theta(n^2)$$

例1.17 求解递推方程 $T(n) = T(2n/3) + 1$

解 上述递推方程中的 $a = 1, b = 3/2, f(n) = 1$, 那么

$$n^{\log_{3/2} 1} = n^0 = 1, \quad f(n) = 1$$

相当于主定理的第二种情况. 根据定理得到.

$$T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

应用实例

例1.18 求解递推方程

$$T(n) = 3T(n/4) + n \log n$$

解 上述递推方程中的 $a=3$, $b=4$, $f(n)=n \log n$, 那么

$$n \log n = \Omega(n^{\log_4 3 + \varepsilon}) = \Omega(n^{0.793 + \varepsilon}), \quad \varepsilon \approx 0.2$$

此外, 要使 $a f(n/b) \leq c f(n)$ 成立, 代入 $f(n)=n \log n$, 得到

$$\frac{3n}{4} \log \frac{n}{4} \leq c n \log n$$

显然只要 $c \geq 3/4$, 上述不等式就可以对充分大的 n 成立.
相当于主定理的第三种情况. 因此有

$$T(n) = \Theta(f(n)) = \Theta(n \log n)$$

不能使用主定理的例子

例4 求解 $T(n)=2T(n/2)+n\log n$

解 $a=b=2$, $n^{\log_b a}=n$, $f(n)=n\log n$

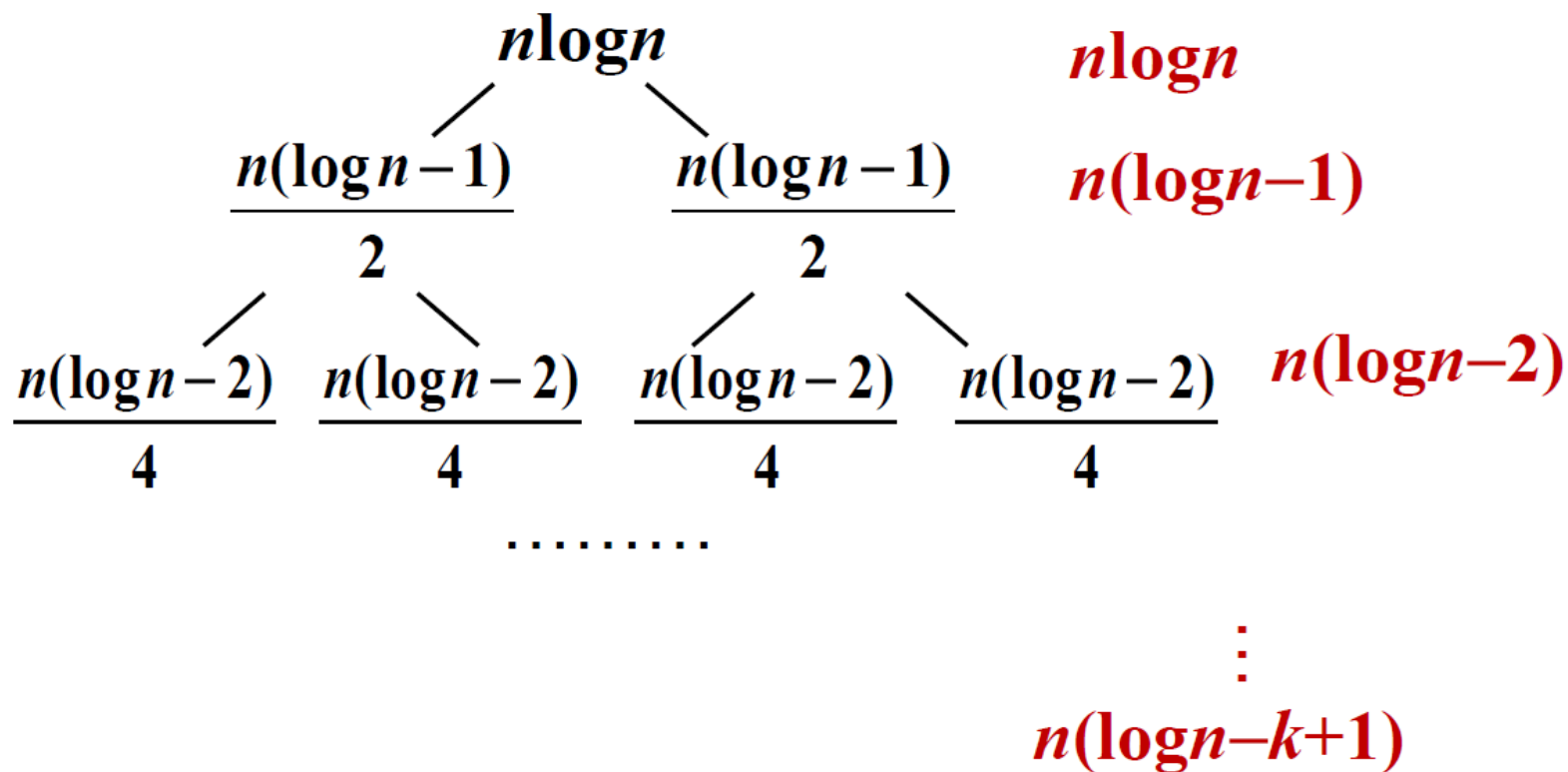
不存在 $\varepsilon > 0$ 使下式成立

$$n \log n = \Omega(n^{1+\varepsilon})$$

不存在 $c < 1$ 使 $af(n/b) \leq cf(n)$ 对所有充分大的 n 成立

$$2(n/2)\log(n/2) = \underline{n(\log n - 1)} \leq cn \log n$$

递归树求解



求和

$$T(n)$$

$$= n \log n + n(\log n - 1) + n(\log n - 2) \\ + \dots + n(\log n - k + 1)$$

$$= (n \log n) \log n - n(\underline{1 + 2 + \dots + k - 1})$$

$$= n \log^2 n - n \underline{k(k - 1) / 2} = O(n \log^2 n)$$